

C++

1. OOPS Concepts

Class, Object, Abstraction, Encapsulation, Inheritance, Polymorphism.

Class: - Class is a **logical construction**, or it is called a blue-print of your object, member variables will not be allocated by defining the class.

Object: - Object is a **physical construction**, member variables will allocate memory.

Abstraction: - **Hiding the implementation details** and **exposing the essential features** is called abstraction. **Abstraction is the designing perspective view**

Abstraction can be achieved in C++, by using interfaces, and abstract base classes. **We are sharing common code or data with derived classes.**

for ex: - class shape, class line, class circle, class rectangle.

Here **Draw ()** and **Serialize ()** is the behavior of these classes.

Here Shape class provides abstraction for drawing and serialization.

Shape class is not providing any kind of implementation for **Drawing** a specific shape and **serializing** the specific dimensions of the particular shape.

class line, circle, and rectangles are having their own implementations for Draw () and Serialize () methods.

C++ STL provides various container classes, Here **Container** is the abstract base class(abstraction) and **Vector, List, and Map classes are the concrete classes.**

Data Hiding: - Defining the member variables in private section. Member variables defined in private cannot be accessed from outside of the class. Private member variables cannot be accessed through objects is called data hiding.

Encapsulation: - **Binding the member variables** and **methods together** in a **single unit** is called encapsulation. Encapsulation is achieved with the private, protected, and public access specifiers. **Defining member variables in private section, and providing get/set accessors (methods) in public section.**

for ex: - obj1.fun ();

Here fun method can access the data of the **obj1** only. **fun ()** method cannot access some other objects data. Calling a class method, through object is called encapsulation.

Encapsulation is all about protecting the data, and Abstraction is hiding the implementation.

Inheritance: - Inheritance is a mechanism of **reusing** and **extending existing classes without modifying them**, it is producing hierarchical relationships between them.

The object being inherited from is called the parent or base, and the object doing the inheriting is called the child or derived object.

Why the need for inheritance in C++?

One of the fundamental ideas behind object-oriented programming is that code should be reusable.

Polymorphism: - Polymorphism means one name many forms. Polymorphism means one object behaving as multiple forms. One function behaves in different forms. In other words, "Many forms of a single object" is called Polymorphism."

2. Difference between Ref Type, and Pointer Type?

Pointer Type	Ref Type
1. Pointers can be NULL. 2. Pointers may not be initialized at the time of its creation. 3. Pointer arithmetic is possible. <pre>int x[10]; int *p = &x[0]; p++; // now p points to x[1] By incrementing the pointer, it moved to the next memory location.</pre> 4. Pointers can be re-assigned.	1. Ref type cannot be NULL. 2. Ref type must be initialized at the time of its creation. for ex: - int &b = a; 3. arithmetic is not possible. Ref types are internally constant pointers. 4. ref types cannot be re-assigned once they are initialized.

FAQ: - When you are going to use pointers?

When you are going to use ref types.?

3. What is abstract class?

If any class is having at least one pure virtual function, then it is abstract class.

- An abstract class can have implemented methods also.
- You cannot create an instance(object) from abstract class.

Purpose of writing an abstract class is to have a single interface for multiple types of objects. Abstract class forces the sub class to implement the pure virtual functions of base classes.

4. What is interface?

We don't have interfaces in C++

But in COM technology we have interface mechanism.

interface IMath

{

int add (int x, int y);

int sub (int x, int y); };

- By default, methods declared in interface are pure virtual functions.
- An interface can have only pure virtual functions.
- You cannot write any implemented method in interface.
- You cannot create any instance from interface.

Interfaces are providing a contract between interface and its sub class.

Interfaces are used to create pointers, which points to the sub class object address.

5. What is initialization list?

class member variables can be initialized using initialization list.

We have two ways to initialize the member variables of the object.

Inside constructor body.

Right side of the constructor header with colon operator (initialization list).

Inside constructor body.	Right side of the constructor header with colon operator (initialization list).
<pre>class Abc { int x; int y; public: Abc (int n1, int n2) { x = n1; y = n2; } };</pre>	<pre>class Abc { int x; int y; public: Abc (int n1, int n2): x(n1), y(n2) { } };</pre>

- initialization list will give good performance.

initializing inside constructor body is...

```
int x;
x = n1;
```

initializing with initialization list is like ...

```
int x = n1;
```

Note: - If class member variables are const or ref type, then they must be initialized with initialization list only.

6. What is mutable?

If any const method is required to modify the class member variables. Those member variables must be defined with mutable modifier.

for ex: - **mutable** int
a;

So, variable 'a' can be modified by const methods of the class.

7. What is volatile?

"volatile" keyword is used to avoid the compiler optimization.

Suppose, I have a global variable... **int a;**

and, I have two threads...

One thread is written as...

```
void fun1(void *gp)
{   a =
1;
    while (a == 1)
```

```

    {
        //...
    }
}

```

Here in this while loop, I'm not modifying the variable 'a', and my expression is (a == 1)

Here compiler optimizes, my expression (a == 1) as while (1)

But I'm modifying this variable 'a', from another thread. So, to avoid the compiler optimization, **int a = 1;** must be defined as...

volatile int a = 1;

8. Explain method overloading and method overriding?

Overloading is method overloading or operator overloading and it is a **compile-time polymorphism**.

Overriding is base class virtual methods are overridden in the derived class with same signature, it is **run-time polymorphism**.

method overloading is same method name by differentiating with no. of args and type of args. here method call will be resolved by the compiler, by checking no. of args and type of args passed.

add (10, 20); // will call add function with two integer args definition. **add (2.3, 5.6);** // will call add function with two double args definition.

compiler uses name mangling technique to resolve the function call. Assigning unique names to each function is called name mangling, it depends on the compiler.

Operator overloading is overloading the existing operators like **+ - * / % =** to perform additional task with user defined objects.

Method Overriding is, base class virtual functions will be overridden in the derived class by writing the same signature.

Here function call will be resolved at runtime by using **VTABLE** mechanism. **Base *bPtr = new Derived;** **bPtr->fun1();** // here **fun1()** method called on base pointer, which points to the derived object.

actually, it should call base method **fun1()**. But at run-time the call will be resolved to call the derived class method.

9. What is Runtime-Polymorphism or Late Binding?

- **Runtime polymorphism is overriding the base class virtual functions in the class hierarchy.**

- method calling will be resolved at runtime by using **VTABLE's**
- **a VTABLE is a static function pointer array, which contains most derived class virtual function addresses.**
- **VTABLE is created at compile-time, and __vfptr assigned with the VTABLE by using the constructor of class.**

for ex: - I have two classes, class **Abc** and class **Xyz**

<pre> class Abc { public: virtual void fun1() { cout<<"Abc::fun1"; virtual void fun2() { cout<<"Abc::fun2"; } }; </pre>	<pre> class Xyz: public Abc { public: void fun1() // overriding { cout<<"xyz::fun1"; } }; </pre>
--	--

Here two VTABLE's will be created.

Abc VTABLE contains...

&Abc::fun1	&Abc::fun2
-----------------------	-----------------------

Xyz VTABLE contains...

&Xyz::fun1	&Abc::fun2
-----------------------	-----------------------

- Here **__vfptr** will be added to the **Abc class**, which contains **Abc VTABLE address**.
- **Abc __vfptr** will be inherited to **Xyz class**, so **Xyz __vfptr contains Xyz VTABLE address**.

If a class contains virtual functions, for that class one VTABLE will be added.

Each object created from that class contains first four bytes will be vfptr. By using vfptr, it resolves function calling.

Abc *ptr = new Xyz; // base pointer points to the derived object address
ptr->fun1(); // **ptr** contains **Xyz object**, **Xyz object** first four bytes is **Xyz's __vfptr**, which contains, **Xyz's VTABLE**, which contains **Xyz's fun1 address**.

By using virtual, we can create either “virtual” methods (or) virtual base classes.

virtual methods: -

- i. virtual methods can be, simple (or) pure virtual methods.
- ii. Simple virtual methods, will have the body, and pure virtual methods will not have the body. **for example: -** `virtual void fun () = 0;`
- iii. If any class have at-least one pure virtual method, it's known as abstract base class, we cannot create any instance.

virtual base classes: -

- i. virtual keyword can be used to create virtual base class, to avoid the diamond problem.

11. Object slicing?

If derived object is assigned to base object, then base portion only will be copied from derived object. derived portion will be sliced away.

<pre>class A { int a; public: A (int x=0) {a = x;} };</pre>	<pre>class B: public A { int b; public: B (int x=0, int y=0): A (x) {b = y;} };</pre>
---	---

B obj1(10,20); A obj2; obj2 = obj1; // here 10 will be copied to obj2 from obj1.

12. Types of inheritances?

Single: - One base class and one derived class.

Multilevel: - A class having one base class and one sub-class.

for ex: -

- class B is derived from class A
- class C is derived from class B.

Multiple: - A class is deriving from more than one base class.

for ex: -

- class C is derived from class A and class B

Hybrid: - combination of Multilevel and Multiple.

A class can have more than one base class and more than one sub class.

Hierarchical: - A class can have only one base class and can have more than one sub class.

for ex: -

- class B is derived from class A
- class B has 3 sub classes Class C, Class D, Class E.

Note: - C++ supports multiple inheritance also. Java, C# languages are supporting only Hierarchical inheritance.

13. Explain type castings in C++?

static_cast: -

- narrow conversions: - int variable assigned to short type variable

- | |
|---|
| Syntax: - <code>static_cast < targettype > (sourcetype);</code> <code>int x =</code>
<code>static_cast < int > (colors);</code> // here colors type is enum
<code>Base *bptr = static_cast < Base *> (&dObj);</code> // here dObj type is Derived. |
|---|

- It allows only up-casting. (derived object address is, assigned to base pointer)
- If you are doing down-casting (base object address is, assigned to derived pointer). it gives compile-time error unless those classes are polymorphic.
- If classes are polymorphic, means..., they have virtual functions. And you are doing down casting it returns NULL.
- It uses RTTI (runtime type identification) to identify the type to be converted.

```
Syntax: - dynamic_cast< target_type > (source_type);           base *bptr =
dynamic_cast<base *>(&dobj); // dobj is derived class object.
// it is successful conversion from derived to base.

derived *dptr = dynamic_cast<derived *>(&bobj); // bobj is base object.
// it gives compile-time error.

suppose base1, derived1 are polymorphic classes having virtual methods.
derived1 *dptr1 = dynamic_cast<derived1 *>(&bobj1); // bobj1 is base object.
// it's not compile time error. dptr1 assigned with NULL.
```

If you want to assign const variable address to non const pointer, we need to apply constant casting.

Syntax: -

```
const int n = 100;
int *ptr = const_cast<int *(&n); // success
```

int *ptr = &n; //
compile-time error.

```
int *ptr = (int *) (&n); // success, but not recommended.
```

Suppose you want to do conversion between two different types of objects, which are not in class hierarchy.

```
Syntax: -      class Abc {};  class
Xyz {};
```

```
Abc *ptr = reinterpret_cast<Abc *>(&obj); // obj is Xyz object.
```

14. Difference between copy constructor and overloaded assignment operator? Both are provided by the compiler by default.

Copy Constructor	Overloaded Assignment operator
Copy constructor called at the time of creating an object from existing object. Abc obj2 (obj1); // copy constructor called. Copy constructor called in 3 scenarios. 1. creating an object from existing object Abc obj2 (obj1); 2. passing object as an argument to any method by value. void fun (Abc obj) { } 3. returning local object from a function. Abc fun () { Abc obj; return obj; }	Overloaded assignment operator will be called, with object assignment, which are already created. Abc obj1, obj2; obj1 = obj2; // overloaded assignment called.
Compiler provided copy constructor and overloaded assignment operator is carrying shallow copy copying, which is value type copying.	
If object is having pointer fields, then shallow copying is not suitable , so we must implement our own copy constructor and overloaded assignment operator to perform copying .	

15. Shallow Copy Vs Deep Copy

Compiler provided copy constructor and overloaded assignment operator is performing **shallow type copying**, shallow type copying is value type copying.

for ex: -

```
int a = 10;
```

```
int b = a;
```

Here b, and a are having its own copies, if variable 'a' is destroyed, it will not effect to the variable 'b'.

If class having member variables as pointers or reference type, then we must provide our own copy constructor and overloaded assignment operator to **perform deep copying**.

for ex: -

```
int *p1 = new int;
```

```
*p1 = 100;
```

```
int *p2 = p1; // this kind of assignment is value type copying
```

for pointers we have to allocate memory dynamically and copy the data.

```
int *p2 = new int;
```

```
*p2 = *p1; // this is deep copying....
```

- If class having member variables as pointers or reference types, and I'm not performing deep copying, then both pointers will have the same data.
- if one object is destroyed, another object member variables will become dangling pointers.

16. How to resolve diamond problem (Virtual base class)?

In Inheritance, diamond problem can be resolved with the virtual base class.

Suppose, I have 4 classes, **class A**, **class B**, **class C**, and **class D**.

B and **C** classes are derived from **class A**. **class**

D is derived from **class B**, and **class C**.

Here, I have a method **fun ()**, in **class A**.

so, **fun ()** method copy is available in **class B**, and **class C**.

class D, is derived from **B**, and **C**. So, **class D** will get **fun ()** method copy from **class B**, and **class C**.

from **class D**, if I'm calling **fun ()** method, we will get ambiguity error... which **fun ()** method to be called, **class B::fun ()** copy, or **class C::fun ()** copy. This is diamond problem.

To resolve the diamond problem, we are making class A, as virtual base class.

class B declaration is... **class B: virtual public A** **class**

C declaration is... **class C: virtual public A**

If **class D** is derived from **B**, and **C**.

B and **C** classes own implemented methods will be inherited to its sub-class.

class A, members are directly inherited to the **class D**, So, we won't get any duplicate methods.

17. Explain about virtual destructor?

In class hierarchy objects are constructed from top to bottom and destruction occurs from bottom to top.

I have two classes, **class Abc** and **class Xyz**. **Xyz** is derived from **Abc**.

<pre>class Abc { public: virtual ~Abc () { } };</pre>	<pre>class Xyz: public Abc { };</pre>
---	--

when an object constructed from **Xyz**, first **Abc** constructor called and next **Xyz** constructor called. when destroying **Xyz's destructor** called first, and **Abc's destructor** called later.

In runtime polymorphism, we are creating **polymorphic classes**. The class which is having **virtual functions** that is **polymorphic class**. In these scenarios we are assigning derived object address to base class pointer.

```
Abc *ptr = new Xyz;
```

delete ptr; // Here **Abc's destructor** only will be called, it doesn't call **Xyz's destructor**, it leads to **memory leaks**, to avoid this situation, we are making the **Abc's destructor as virtual**. so... **Xyz's destructor** called first, and **Abc's destructor** called later.

18. What is virtual constructor?

There is no virtual constructor. If any class is having virtual methods, for that class `__vfptr` will be added and also VTABLE will be created. VTABLE contains most derived class virtual function addresses.

- **`__vfptr`, and VTABLE are added to the class at compile-time.**
- **VTABLE is assigned to the `__vfptr` by constructor of the class.**

Here constructor responsibility is **assigning VTABLE address to `__vfptr`**.

So, we cannot delegate the constructor functionality to other class. We cannot make the constructor as virtual.

19. What is explicit constructor?

Argument constructors must be called explicitly, they are behaving like conversion constructors. To avoid this, we are making argument constructors as explicit constructors using **explicit** keyword.

Suppose I have a **class Abc**. class

```
Abc
{
public:
    Abc (int x) { };
    void fun (Abc obj) { };
};
```

If I am invoking **fun (Abc obj)** method by **passing value 100 as parameter**.

100 will be converted as Abc object by invoking the constructor which takes int as parameter.

So, I don't want my **int parameterized constructor to behave like conversion constructor**. So, I am putting **explicit keyword in the constructor definition**.

```
explicit Abc (int x);
```

So, invoking of **fun ()** method with int value **will give compile time error**.

20. What is dangling pointer?

If we are trying to access a pointer, which is already deallocated, that **pointer is called as dangling pointer.**

```
int *ptr1 = new int;
*ptr1 = 100; int
*ptr2 = Ptr1;
delete ptr1;
```

int x = *ptr2; //Here ptr2 is dangling pointer.

21. What is memory Leak?

Memory allocated dynamically is not deallocated and we lost the handle of that memory.

```
void fun ()
{   int *ptr = NULL;
    ptr = new int;
    *ptr = 100;
}
```

- Here 4 bytes of memory allocated on heap and that address stored in ptr.
- if execution flow is moving out from this local scope, ptr variable will be destroyed. ○ Heap 4 bytes will not be destroyed yet and we lost that address also.

We have a leak detector tool, which is provided by our client. If I'm loading .exe by using this tool, it will create a log file which contains memory leak details. in which function memory leak happens.

22. How do you find memory leaks? Without using tool?

Detecting memory leaks are handled by the **debugger and the C Run-Time Libraries (CRT) debug heap functions.**

To enable the debug heap functions...

```
#define _CRTDBG_MAP_ALLOC // define macro
#include<stdlib.h> // include header files
#include<crtdbg.h>
```

Call... **_CrtDumpMemoryLeaks ()** function before an **application exit point**, to display a **memory-leak report** when your application exits.

If my application has several exit points, then...

A call to **_CrtSetDbgFlag (_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF);**

at the **beginning of your application** will cause an automatic call to **_CrtDumpMemoryLeaks ()** function at each exit point.

23. What is Heap corruption?

Heap corruption occurs when dynamic allocation of memory is not handled properly.

for example: -

- reading or writing data to outside the bounds of array.

- calling delete statement, more than once on a same pointer.
- incorrect type casting of dynamic data.
- access to uninitialized memory (which are wild pointers)
- a program calls **free ()** on addresses that were not returned by **malloc**, such as static, global, or automatic variables, or other invalid expressions.

24. What is exception handling?

Exception is an unexpected situation at runtime.

- If you are not handling the exception, the application **will be crashed**, and you don't have any hint.
- **If you are suspecting any statement** in the program may **cause runtime errors**. **Those statements must be written in the try block and handle the exception details in catch block.**

Examples: -

- divide by zero will cause runtime error (Its hardware error by ALU)
- If you are opening a file to write, which is write protected.
- If you are converting a string type to int, which is incompatible.

points to remember while handling exceptions are...

- Each try should have catch block
- A try can have more than one catch block.
- catch(...) is used to handle any type of exception.
- throw keyword will throw the exception to the nearest catch block.

25. What is stack unwinding?

When an exception is thrown from a try block, then control passes to a nearest catch handler, **the C++ run time calls destructors for all automatic objects constructed since the beginning of the try block. This process is called stack unwinding.**

The automatic objects are destroyed in reverse order of their construction.

- stack unwinding process should never throw an exception.
- if you are suspecting your destructors may throw an exception, put your destructor block statements in try { and } catch(...) {}

26. How to create non-inheritable class?

- In class hierarchy constructors are called from base to derived.
- If, I'm creating a class with private constructors, the class itself not able create objects.

Steps to create non-inheritable **class Abc**

Create **DummyClass** with default constructor in private section.

Make **Abc class** is friend to the **DummyClass**.

Derive **Abc class** from **DummyClass**, by making **DummyClass** as **virtual base class**.

So, if any one deriving from **Abc class**, that derived class is also having the **DummyClass** as **base class**, because **DummyClass** is virtual base class to the **Abc class**.

So, **Abc sub-class is not able to create objects**, because of **DummyClass private constructor**.

27. What is a template?

template is a mechanism of code (algorithm) re-use. Template programming is called generic programming.

Template type or generic type allows you to create function templates or class templates.

A function defined with **template type parameter**, can be called without regarding its **parameter type**.

function template code: -

```
template<class T>
T add (T a, T b)
{
    return a + b;
}
```

Suppose I'm calling...

- **add (n1, n2);** // n1, n2 are integer types.
- **add (d1, d2);** // d1, d2 are double types

Here compiler will overload **add ()** function with 2 integers arguments and 2 double argument types.

Advantage: - Reduces the re-writing of the code (or) **code re-use**.

Disadvantage: - Code bloat (binary size will be increased after compilation). **class**

template code: -

<pre>template<class T> class Node { T data; Node<T> *next; };</pre>	<pre>template<class T> class List { Node<T> *first; Node<T> *last; public: List () {} void add (T data) {} void display () {} };</pre>
---	--

```

void main () // client code
{
    List<int> list1; // creates a collection to hold int types.
    List<double> list2; // creates a collection to hold double types.
}

```

28. What is template specialization?

Template functions can be overloaded to work for different kinds of data types.

Suppose...

I have written a function template to swap two variables.

Suppose...

I'm calling swap function by passing value type variables like int, long, float or double ... it works fine.

But this swap template function will not work for char pointer types.

So, we need to write another function template which works for char pointer types also.

```

template<class T> void
swap (T &a, T &b)
{ // swapping logic }

```

```

template<> void swap (char *&ptr1, char *&ptr2) // It is a pointer to a
reference type. { // swapping logic }

```

29. what is smart pointer (auto_ptr)?

- smart pointers are used to avoid memory leaks.
- dynamically allocated memory on heap is referred by smart pointers.
- smart pointers will call the destructor, while it is moving out from the local scope. built-in

smart pointer is **auto_ptr**. (which is template type) Syntax: -

```

void fun ()
{
    auto_ptr<Abc> ptr (new Abc);
    ptr->display ();
}

```

Here when **ptr** is going out from local scope automatically it calls **Abc's destructor**.

Disadvantage: - If **auto_ptr** pointer is assigned to another **auto_ptr** pointer, then life time also Will be transferred to the target pointer, it cannot be shared.

So, **auto_ptr** is

obsolete in C++ 11

How can you implement your own smart pointer class?

We can write our own smart-pointer class also by overloading...

*** operator**

-> operator

= operator

-> and * operators will return the reference of the object

= operator will increase the reference count of the object.

30. Explain smart pointers of C++ 11?

Smart pointer `auto_ptr` is obsolete and should no longer be used.

C++11 introduces 3 types of smart pointers `unique_ptr`, `shared_ptr` and `weak_ptr`.

unique_ptr: - `std::unique_ptr` was developed in C++11 as a replacement for `std::auto_ptr`.

`std::unique_ptr` is a **smart pointer** that owns and manages another object through a pointer and **disposes of that object when the `unique_ptr` goes out of scope.**

Here Resource is maintained by only one pointer.

- No copy constructor or copy assignment (overloaded assignment)
- Copying or assignment handled by **move constructor or move assignment.**

destruction: - `unique_ptr` destroys object either of the happens...

1. `unique_ptr` goes out of local scope
2. `unique_ptr` is assigned to another pointer via **move construction or move assignment** or **reset ()**.

There are two versions of `unique_ptr`

1. **Manages a single object** (e.g. allocated with `new`)
2. **Manages a dynamically-allocated array of objects** (e.g. allocated with `new []`)

When to use `unique_ptr`?

Use `unique_ptr` when you want to have single ownership (Exclusive) of resource.

Syntax: -

```
std::unique_ptr<int> p1 = std::make_unique<int> (100);  
std::unique_ptr<int> p2 = std::move(p1); // p1 copied to p2, p1 becomes null
```

shared_ptr: - `std::shared_ptr` is a **smart pointer** that retains **shared ownership of an object through a pointer.**
Several `shared_ptr` objects may own the same object.

Here Resource is maintained by several pointers.

It is a reference counting ownership model i.e. the counter is incremented each time a new pointer points to the resource and decremented when destructor of object is called.

destruction: - `shared_ptr` destroys object either of the following happens

- last remaining **shared_ptr** goes out of local scope.
- last remaining **shared_ptr** is called **reset ()**.

When to use shared_ptr?

Use **shared_ptr** if you want to **share ownership of resource**. Many **shared_ptr** can point to single resource. **shared_ptr** maintains reference count for this purpose. when **all shared_ptr's pointing to resource goes out of scope the resource is destroyed**.

Syntax: -

```
std::shared_ptr<int> p1 = std::make_shared<int> (100);
std::shared_ptr<int> p2 = p1;

cout << "p1: " << *(p1.get ()) << endl;
```

weak_ptr: - **std::weak_ptr** is a smart pointer that holds a non-owning ("weak") reference to an object that is managed by **std::shared_ptr**.

It must be converted to std::shared_ptr in order to access the referenced object.

It is required in some cases to **break circular references** between **shared_ptr** instances.

Cyclic Dependency (Problems with shared_ptr):

Let's consider a scenario where we have two classes A and B, both have pointers to other classes. So, it's always be like **A is pointing to B and B is pointing to A**. Hence, **use_count** will never reach zero and they never get deleted.

Syntax: -

```
std::shared_ptr<int> p1 = std::make_shared<int> (42); std::weak_ptr<int> gw
= p1; // creates weak reference to the shared pointer.

if (auto spt = gw.lock())
{ // Has to be copied into a shared_ptr before usage
    std::cout << *spt << "\n";
}
```

31. Explain various stages of compilation of code to produce .exe file?

Preprocessing is the first pass of any C compilation. It processes include-files, conditional compilation instructions and macros.

Compilation is the second pass. It takes the output of the preprocessor, and the source code, and generates assembler source code.

Assembly is the third stage of compilation. It takes the assembly source code and produces an assembly listing with offsets. The assembler output is stored in an object file.

Linking is the final stage of compilation. It takes one or more object files or libraries as input and combines them to produce a single (usually executable) file. In doing so, it resolves references to external symbols, assigns final addresses to procedures/functions and variables, and revises code and data to reflect new addresses (a process called relocation).

32. Explain types of containers in STL?

Sequence (array and linked list): - **vector, list, deque**

Associative Sequence (binary tree): - **set, map, multiset, multimap**

Un-ordered Associative Sequence (hash table): - **vector_set, unordered_map, unordered_multiset, unordered_multimap**

Adaptors: - **stack, queue, priority_queue**

a) What is internal implementation of vector, list, map, and set vector:

- dynamically growing array.

list: - doubly linked list

map (and) set: - red/black tree data structure **unordered_set**

and unordered_map: - hash table

Note: - c++11 introduced **std::forward_list**, which is a singly linked list.

33. Types of iterators?

1. **Random Access Iterator** **vector**, **deque**, **array** provides random access iterator **vector<int>::iterator itr**; `itr = itr + 5;`
`// advance itr by 5`
`itr = itr - 4;`
if (`itr > itr2`) ... compare position of the iterator
`++itr;`
`--itr;`
2. **Bi-directional iterator:** - can move forward or backward **list**, **set/multiset**, **map/multimap**
`++itr;`
`--itr;`
3. **Forward iterator:** - only forward direction **forward_list** is a single linked list.
`++itr;`
Unordered containers provide at least forward iterator.
4. **Input iterator:** - read and process values while iterating forward
`int x = *itr;`
5. **Output iterator:** - output values while iterating forward
`*itr = 100;`

34. Explain about STL Vector

- Vector is a sequential container class in STL.
- **It is a dynamically growing array, with contiguous memory blocks.**
- By default, initial capacity is zero, based on elements insertion, Pre-allocates space for future elements, so extra space required beyond what's necessary for the elements themselves.
- Insertions at the end are constant, but insertions elsewhere are a costly, **time complexity is O(n).**
- **Insertions and deletions are expensive in the middle.**
- **You can randomly access its elements.**

35. How STL Vector implemented internally for automatic resizing?

Vector is implemented as a dynamically growing array. The memory for this array is allocated in the constructor. As more elements are inserted the array is dynamically increased in size. A constructor without parameter creates an array with a default size. Another constructor with integer parameter creates an array of the specified size. The destructor deletes the memory allocated for the array.

```
template <typename T>
void vector<T> :: push_back (const T &t)
{
    // if we've run out of space, allocate more memory
    if (size_ == reserved_size_)
        resize (reserved_size_ + _DEFAULT_VECTOR_SIZE);

    // size_ when used as an index, points to the next position after
    // the last element in the vector
    array_[size_] = t;

    // now there is one more element in the vector, so increase the size
    size_++;
}

template<typename T>
void vector<T> :: resize(int n) // make the size of the internal array exactly n
{
    if (n > reserved_size_) // if requested size is more than the current size
    {
        // allocate a new array of larger capacity
        // copy the old array to new array
        // destroy old array
        auto_array<T> new_array (new T[n]);
        for (int i=0; i<size_; i++)
        {
            new_array[i] = array_[i];
        }

        delete [] array_;      array_
        = new_array.release ();
        reserved_size_ = n;
    }
}
```

36. Difference between Vector and List?

Vector	List
Vector and List both are sequential containers defined in STL.	
Contiguous memory.	Non-contiguous memory.
Each element only requires the space for the element type itself (no extra pointers).	Each element requires extra space for the node which holds the element, including pointers to the next and previous elements in the list.

Insertions at the end are constant, but insertions elsewhere are a costly $O(n)$.	Insertions and deletions are constant, anywhere in the list.
You can randomly access its elements.	You cannot randomly access elements. so, getting at a particular element in the list can be expensive to iterate from head to tail.
If my container expects random access using index, then I will go with vector	If my collection expects lot of insertions and deletions, in the middle, then I go with list.
You can apply random access iterators	Only bi-directional iterator.

37. Explain about Map and Multi-Map in STL **map**, **multimap**, **set** and **multiset** are associative containers in STL.

- `std::map<int, string>` contains data in the form KEY-VALUE pair collection.
- Keys are inserted as Red-Black Tree data structure.
- So, Searching is very fast compared to other containers.
- Each key associated with one value.
- Key's doesn't have duplicate keys.
- Time complexity of search is: $O(\log n)$

Multimap will have duplicate keys.

Set is a collection of key's only, and they have only unique keys. **Multiset** is like a set but it has duplicate keys.

38. Explain about **map** and **unordered_map** in STL (C++ 11) **unordered_map<key, value>** object is an associated container that stores elements as Key/Value pairs. Internally **unordered_map** is implemented using Hash Table, the key provided to map are hashed into indices of hash table that is why performance of data structure depends on hash function.

Time complexity of (insert, delete, search): $O(1)$

39. Explain `strcpy`, `memcpy`, and `memmove`?

strcpy copies character from source to destination one by one until it finds NULL or '\0' character in the source. `strcpy` is used for strings.

memcpy copies data (not character) from source to destination of given size `n`, irrespective of data in source. `memcpy` is used for encrypted data or binary data

memmove is similar to **memcpy ()** as it also copies data from a source to destination. **memcpy ()** leads to problems when **source and destination addresses overlap** as `memcpy ()` simply copies data one by one from one location to another **which causes data loss**. Where as `memmove` uses a **temporary array** and **copies the data without loss**.

40. Explain composition, aggregation and association?

Composition: - Creating a complex object, from simple objects. Composite object is responsible for destroying the child objects. **for ex:** -

<pre> class PC { Motherboard m_mb; CPU m_cpu; RAM m_ram; }; </pre>	<pre> void main () { PC obj; } </pre>
--	---

Here PC object is destroyed, then all member objects also will be destroyed.

Aggregation: - Creating a complex object from simple objects. Composite object is not responsible for destroying the child objects. child objects may be members to the other composite objects also.

for ex: -

<pre> class Faculty { }; </pre>	<pre> class EEE { Faculty *m_pFac; }; </pre>	<pre> class ECE { Faculty *m_pFac; }; </pre>	<pre> class CSE { Faculty *m_pFac; }; </pre>
<pre> void main () { Faculty *pFaculty = new Faculty; } </pre>			

Here **Faculty object** is member to each class type **EEE**, **ECE** and **CSE**. If I'm destroying **CSE** object the faculty object will alive in **ECE** or **EEE** objects.

Association: - Association uses “**has-a**” relationship between two objects. In other words, association defines the multiplicity between objects. You may be aware of one-to-one, one-to-many, many-to-one, many-to-many all these words define an association between objects.

- Aggregation is a special form of association.
- Composition is a special form of aggregation.

Generalization: - Generalization uses a “**is-a**” **relationship** from a specialization to the generalization class. Common structure and behavior are used from the specialization to the generalized class. At a very broader level you can understand **this as inheritance**.

41. What is the difference between **struct** and **class**?

By default, **struct members are public**, and **class members are private**. Whatever we can do with class like constructors, destructors, methods, virtual methods, everything can be defined with the struct also.

But classes are representing real world objects, and struct are representing technical objects.

Class object examples are like: - customer, supplier, invoice etc.,

Struct object examples are like: - Point object with x, and y axis. And complex objects to represent the mathematical values.

In C# language structs are value types (allocated on STACK), and classes are reference types (allocated on HEAP).

42. Difference between **procedure oriented and **object oriented**? C language is Procedure oriented.**

- Procedure oriented will focus on writing algorithms (functions).
- Data move freely among global functions, and no data security.
- Maintenance of application is very costly.

C++ language is Object oriented

- Everything is object.
- We are defining the objects(data) and methods are associated with object.
- data can be hidden in objects; no outside world function cannot access the private data of class.
- Maintenance is easy, it's like building block type of programming.

43. Explain Memory layout of “C” Program?

A typical memory representation of C program consists of following sections.

Stack: - automatic storage class, will allocate on stack section.

Heap: - malloc, calloc, realloc, or new operator will allocate on Heap segment

Initialized data segment: - Initialized static, and global variables will allocate on Initialized Data section.

Uninitialized data segment: - Uninitialized static, and global variables. **TEXT**

Segment: - coding statements like, printf(...), and scanf(...)

44. Explain about storage classes?

- Storage classes are automatic, static, extern and register.
- Storage class specifies, where the memory will be allocated for the variable, initial value of the variable, what is life time, and accessibility of the variable.

Automatic: -

Life time – with in the function only

Keyword- auto

Visibility- local

Storage- stack

Initial value- garbage value

Static: -

Life time – throughout the application

Keyword- static

Visibility- File Scope or within the defined block

Storage- Data Section

Initial value- zero

Extern: -

Life time – throughout the application

Keyword- extern

Visibility- global

Storage - Data Section

Initial value – zero

Register: -

Life time – with in the function only

Keyword- register

Visibility- local

Storage- CPU register or stack

Initial value- garbage

How to create a file scope variable?

static int a; // it is declared globally.

- Global static variable is a file scope variable.
- We can access global static variable within the file only.

45. Difference between malloc and new?

- **malloc ()** and **new** both are **dynamic allocation of memory i.e. at run time**.
- **malloc ()** is a function, while **new** is an operator.
- **We can't overload malloc** function, **we can overload new** operator.
- If we **allocate memory dynamically** by using **malloc**, we have to **deallocate by using free ()** function.
- While in case of **new**, we have to **deallocate by using delete operator**.
- **malloc will not call constructor** if we create class object using malloc. While in case of "**new**", class **constructor will be called**.
- **malloc** returns **void pointer**, we have to **carry the type casting**, while in case of **new**, **no need of type casting**.
- If memory allocation failed, **malloc ()** function **returns NULL**, and **new** operator **throws an exception (bad_alloc exception)**.
- Using malloc, there is need to include **stdlib.h** header file, no need to include any header file in case of new.

46. How delete [] knows the size of array, to delete all elements?

When you allocate memory on the heap, your allocator will keep track of how much memory you have allocated. This is usually stored in a "**head**" segment just before the memory that you get allocated. That way when it's time to free the memory, the **de-allocator** knows exactly how much memory to free.

47. **Explain about delete this; delete** operator must be used to **deallocate the heap object only**. Here we have two scenarios... **scenario #1 delete this;** called from a method, where current object created on heap using new operator. Here you, should not access any field, **after delete this**. **scenario #2 delete this;** called from a method, where object created on stack, **it is undefined behavior**.

48. Difference between malloc () and calloc ()?

malloc	calloc
malloc () and calloc () both are dynamic allocation of memory i.e. at run time.	
malloc allocated memory block contains garbage values.	calloc initializes the allocated memory to 0
Allocates a block of memory	Allocates multiple blocks of memory , like an array.
ptr_var = (cast_type *) malloc (Size_in_bytes);	ptr_var = (cast_type *) calloc (no_of_blocks , size_of_each_block);
malloc () and calloc () both are required type casting for allocated block.	

49. Difference between macro and function?

macro and inline both are used for code replacement. macro and inline both are improving the performance of execution.

macro	Inline function
It's a preprocessor command , it can be evaluated at pre-Compilation time	Inline functions are processed by the compiler .

Syntax: #define <macro name> <macro expression> For ex: - #define COUNT 100 The pre-processor replaces the macro 'COUNT' in the program with 100.	inline void function_name () {// function body} Inline function body is inserted in place of the function call statement during the compilation process.
	If we declare a function as inline, compiler may or may not treat that function as inline.
Disadvantages: - • Macros are not suitable for arithmetic operations. • Macros simply substitutes value without performing calculation. • Macros type checking is not there	Advantages: • By default, all functions which are implemented inside a class are inline. • Can perform all arithmetic operations. • Inline functions provide type checking Disadvantages: • If inline function having any loop then compiler won't treat as inline.

50. Explain about extern "C" **extern "C" is a keyword to avoid the name mangling or decoration.** Name mangling means unique names are given to the different versions of an overloaded function.

- Name mangling is compiler dependent. Various compilers may provide different name mangling functions.
- Compiler mangles the name as per type, order and no. of arguments.

Global functions or class methods, compiled by the C++ compiler will be decorated with name mangling technique.

So, we must avoid the name mangling of functions, if any C++ function called by the "C" programs.

In my project, we have a 3rd Party "C" DLL "cmosub32". While writing the callback functions, my callback functions should not be mangled, otherwise we will end up with linker error.

for ex: -

```
pa "C" void user_cashflows (...) // this is my callback function, which is called by
{ }                               // icmo_cashflows (, user_cashflows, ...) function
```

51. Explain about functors (function objects)?

A **functor (or function object)** is a C++ class that acts like a function. **Functors** are called using the same old function call syntax. To create a **functor**, we create an object that **overloads the operator ()**.

Advantages of **functors** over the function pointers: - 1. Functors can maintain the state of the object.

3. <https://www.cprogramming.com/tutorial/functors-function-objects-in-c++.html>

52. Explain SOLID Principles?

Single responsibility principle: - A class should have one and only one reason to change, meaning that a class should have only one job.

for example: - **SQLConnection object** will establish a connection to the database. Here SQLConnection class has only one responsibility to **"open a database connection"**.

Similarly, SqlDataReader, and SqlWriter objects.

Open closed principle: - “classes should be open for extension but closed for modification”.

It is the example of inheritance, **CView** class was written by one developer, and it should not be modified, so another developer can extend it as **CFormView** and **CListView**.

Liskov's Substitution principle: - “Parent classes should be easily substituted with their child classes without changing the Parent Class Behaviour”.

for ex: - A square is a specialized Rectangle, **the relationship in between Square and Rectangle is “is a” relationship**. Suppose class Square derived from Rectangle. **If a Rectangle has a reference of Square object. If you're calling SetLength (), and SetWidth () on Rectangle ref of Square object, it will not make sense. So, Square derived from Rectangle will not satisfy the Liskov's Substitution principle**, and it will become bad abstraction on Rectangle and Square.

Interface-segregation principle (ISP) “states that no client should be forced to depend on methods it does not use.”

- Correct abstraction is the key to Interface Segregation Principle.
- When a client depends on methods it doesn't use, it means that your abstractions are wrong.

for ex: -

- ✦ **IMath** interface has two methods, **add (int, int)** and **sub (int, int)**
- ✦ **IScientific** has one method, **GetSinTeta (double);**
- ✦ **IStatistics** has one method, **GetStdDeviation (...);**

Here IMath is the base interface for basic mathematics, IScientific is derived from IMath. If you're deriving IStatistics from IScientific, is not a good abstraction, where statisticians are not interested to do the scientific calculations. Here ISP forces IStatistics must be derived from IMath only.

Dependency Inversion Principle “classes should depend on abstraction but not on concretion”. class Manager, class Designer, and class Tester... these are called concrete classes. If these classes are derived from class Employee, then these classes will have an abstraction. If a Manager class is maintaining a collection of Designers, and Testers... It is not implementing Dependency Inversion Principle. Suppose Manager class need to maintain QA collection, then by defining QA class, we need to update the Manager class as well.

Dependency Inversion Principle consists of two parts:

- 1)High-level modules should not depend on low-level modules. Both should depend on abstractions.**
- 2)Abstractions should not depend on details. Details should depend on abstractions.**

Here in this example Manager is the High-level module, it should not depend on QA class. Both should depend on abstraction.

Now, the code will be changed as Manager class maintains employee collection. Designer, Tester, and QA classes are derived from Employee class.

53. Explain Lambda expression?

In C++11 and later, a lambda expression often called a lambda **is a convenient way of defining an anonymous function, where it is invoked or passed as an argument to a function.**

[capture] (parameter list) {function body}

1. **One pair of square brackets, captures external variables (value type/ref type)**
2. **One pair of parentheses for parameter list**
3. **One pair of curly braces, which is function body.**

Advantages:- (1).Like an inline function, there is no function stack, value pushing on stack, and clean-up of function stack. (2).Improves performance, code readability.

54. Difference between lock_guard and unique_lock? And condition variable?

lock_guard: - When a **lock_guard** object is created; it attempts to take ownership of the mutex it is given. When control leaves the scope in which the **lock_guard** object was created, the **lock_guard** is destructed and the mutex is released. **lock_guard class is non-copyable.**

```
{  
    mutex mtx;  
    std::lock_guard<mutex> lg(mtx); // lg will call mtx.lock ()  
    // some exception may be thrown here.  
} // when 'lg' leaves local scope, lg will call mtx.unlock ()
```

unique_lock: - It has the same interface as **std::lock_guard** but provides additional methods for explicitly locking and unlocking mutexes and deferring locking on construction. **unique_lock class is a movable, but not copyable.**

- By passing **std::defer_lock** instead of **std::adopt_lock** the mutex remains unlocked, when a **std::unique_lock** instance is constructed.
- The lock can then be obtained later by calling **lock ()** on the **std::unique_lock** instance or alternatively, by passing it to the **std::lock ()** function.
- To check if a **std::unique_lock** currently owns its associated mutex the **owns_lock()** method can be used.
- The ownership of a **std::unique_lock** can be transferred between instances.
- Keeping **std::unique_lock**'s additional lock status up-to-date induces some additional, minimal space and speed overhead in **comparison to std::lock_guard**.
- Hence, as a general rule, **std::lock_guard** should be preferably used when the additional features of **std::unique_lock** are not needed.

Condition variable: - A **condition variable** is an object, able to **block** the calling **thread** until notified to resume.

It uses a **unique_lock (over a mutex)** to lock the thread when one of its **wait functions** is called. The thread remains blocked until woken up by another thread that calls a **notification function** on the same **condition_variable** object.

(1) std::mutex mtx; std::condition_variable cv; std::unique_lock<std::mutex> lck(mtx);	(2) scenario #1 cv.wait(lck); // waits, until condition variable gets notification #scenario #2 cv.wait (lck, [] {return ready;})// waits, until second parameter returns true.
(3) cv.notify_all (); // giving notification to all waiting threads.	

http://jakascorner.com/blog/2016/02/lock_guard-and-unique_lock.html

55. What is structure padding in c?

// To do

56. C++ optimization techniques?

// to do

57. Difference between struct and union?

Both are used to define user defined type.

Struct	Union
struct will allocate space for all of its members for example: - struct emp { int m_eno; char m_ename[80]; double m_length; }; Here struct emp size is 92 bytes.	union will allocate space for its highest size member variable, for example union emp { int m_eno; char m_ename[80]; double m_length; };
	Here union emp will allocate for its member m_ename, and it is shared with another member variable.

We have some technical objects, where union types are needed. **for example: -** while taking the dimensions of any object, for example Box object dimensions length and breadth. Here my measurements can be in inches or centimeters or meters. So... **struct Box**

```
{
union {
    float meters;
    double centimeters;
    long inches;
} length, breadth;
};
```

58. What is lvalue, rvalue, lvalue reference (reference type) and rvalue reference (introduced in c++11)

lvalue: - An object that occupies some identifiable location in memory.

rvalue: - An object that is not lvalue.

examples of lvalue int a = 10; // 'a' is lvalue int *p = &a; // a's address is identifiable a = 10; // memory content is modified class Dog; Dog d1; // d1 is lvalue of user defined type (class) // most variables in c++ code are lvalues	examples of rvalue int x = 10; // 10 is rvalue x = a + 5; // (a+5) is rvalue int *p = &(a+3); // error a+2 = 4; // error 2 = a; // error Dog d1; d1 = Dog (); // Dog () is rvalue of user defined type (class) int sum(int x, int y) { return x + y;} int l = sum(3,4); // sum(3,4) is rvalue
rvalues: - 2, a+2, Dog (), sum (3,4), x+y lvalues: - x, a, d1, p	
Summary 1. Every C++ expression yield either a rvalue or a lvalue. 2. If the expression has an identifiable memory location address, it's a lvalue; otherwise rvalue.	

Reference (or lvalue reference) int

a;
int &r = b; // r is lvalue reference int &r = 5; // error, lvalue
reference cannot be assigned with rvalue.

const int &r = 5; // constant lvalue reference can assigned a rvalue.

int square (int &x) {return x * x;} square
(i); // OK
square (5); // error
int square (const int &x) {return x * x;} // **square(i)**, and **square (5)** is OK

What is rvalue reference?

int a = 10; // 'a' is lvalue int &b = a; // 'b' is lvalue reference (reference type) int &&c; // 'c' is rvalue reference	
--	--

59. Compiler generated functions to an empty class?

class Abc { // C++ 03: Abc (); // default constructor Abc (const Abc&); // copy constructor Abc& operator= (const Abc &); // copy assignment operator ~Abc (); // destructor // C++ 11 Abc (Abc&&); // move constructor Abc& operator= (Abc &); // move assignment operator };
--

```
class Abc {};
```

```
// C++ 03
```

1. Default constructor (generated only if no constructor declared by user)
2. Copy constructor (generated only if, 5, 6 not declared by user)
3. Copy assignment operator (generated only if, 5, 6 not declared by user)
4. Destructor

```
// C++ 11
```

5. Move constructor (generated only if, 2,3,4, 6 not declared by user)
6. Move assignment operator (generated only if, 2,3,4, 5 not declared by user)

60. What is tuple?

A tuple is an object that can hold a number of elements. The elements can be of different data types. The elements of tuples are initialized as arguments in order in which they will be accessed.

get<> function is used to access the elements

```
tuple<int, string, double> t1 = make_tuple (101, "Nagesh", 95000.00); cout << get<0>(t1) << endl; cout << get<1>(t1) << endl; cout << get<2>(t1) << endl;
```

```
get<0>(t1) = 105; // get returns a reference of '0' index element.
```

61. C++ 11 Features // To do

62. C++ 14 features

What is struct padding?

What is Hash and Digest?

What is Hashtable, What is Hashing How

to debug template functions?

Explain about operator precedence?

Can I use, pre, and post increment operator at the same time?

For ex:- ++a++

Execution plan in SQL Server Management studio? How

to check or troubleshoot a query execution?

Design Patterns

In my current project I involved with Singleton, Factory, Bridge and Observer design patterns.

1. How to create thread-safe Singleton class?

Singleton class is a creational design pattern.

Singleton class responsibility is to create only one instance for the entire application, and that instance must provide global point of access.

```
class MySingleton
{
private:
```

```

        static MySingleton * m_pOnlyOneInstance;
        static CRITICAL_SECTION m_CriticalSection;
static bool m_bInitializeCriticalSection;
        //holds one and only object of MySingleton
        MySingleton(){} // private constructor
        MySingleton(MySingleton &obj) {} // private copy constructor

public:
        //static method that returns only one instance of MySingleton
static MySingleton * GetInstance()
    {
        if(!m_bInitializeCriticalSection)
            InitializeCriticalSection(&m_CriticalSection);

        if (m_pOnlyOneInstance == NULL) // if not yet instantiated
        {
            EnterCriticalSection(&m_CriticalSection); // allows one thread

            if (m_pOnlyOneInstance == NULL)
            {
                m_pOnlyOneInstance = new MySingleton();
            }

            LeaveCriticalSection(&m_CriticalSection);
        }
        return m_pOnlyOneInstance;
    }

};

MySingleton * MySingleton::m_pOnlyOneInstance = NULL;
CRITICAL_SECTION MySingleton::m_CriticalSection; bool
MySingleton::m_bInitializeCriticalSection = false;

int main( )
{

    MySingleton *pInstance = MySingleton::GetInstance();

    return 0;
}

```

2. Explain about Factory design pattern?

- creates objects without exposing the instantiation logic to the client.
- refers to the newly created object through a common interface.

For ex: - class **Vehicle**, class **TwoWheeler**, class **ThreeWheeler** and **Factory** class.

Here **Factory class** is designed to create instances for **TwoWheeler** and **ThreeWheeler** classes.

TwoWheeler and ThreeWheeler objects are having the common interface which is Vehicle.

```
enum vehicle_type{VT_TWO_WHEELER, VT_THREE_WHEELER};
```

```

class Vehicle
{
    public:
        virtual void print_vehicle( ) = 0; };

class TwoWheeler: public Vehicle
{
    public:
        void print_vehicle()
        { cout<<"Two Wheeler"<<endl; }
};

class ThreeWheeler: public Vehicle
{
    public:
        void print_vehicle()
        { cout<<"Three Wheeler"<<endl; }
};

class Factory
{ public:    static Vehicle *
Create(vehicle_type type)
    {
        Vehicle *pVehicle;

        if (type == VT_TWO_WHEELER)
pVehicle = new TwoWheeler();
        else if (type == VT_THREE_WHEELER)
            pVehicle = new ThreeWheeler();

        return pVehicle;
    }
};

int main( )
{
    Vehicle *pVehicle1 = Factory::Create(VT_TWO_WHEELER);
    pVehicle1->print_vehicle();

    Vehicle *pVehicle2 = Factory::Create(VT_THREE_WHEELER);
    pVehicle2->print_vehicle();

    return 0;
}

```

Coding Practise

Write a program to reverse a single linked list.

<pre> #include<iostream> using namespace std; class Node { int data; Node *next; friend class List; }; class List { Node *first, *last; public: List () { first = last = NULL; } void Add (int n) { Node *curr = new Node; curr->data = n; curr->next = NULL; if (first == NULL) first = last = curr; else { last->next = curr; last = curr; } } int main() { List obj; obj.Add(10); obj.Add(20); obj.Add(30); obj.Add(40); obj.display(); obj.reverse(); cout<<"\n reverse :"; obj.display(); return 0; } </pre>	<pre> /* reverse of linked list logic */ void reverse () { Node *prev = NULL; Node *current = first; Node *next; while (current != NULL) { next = current->next; current->next = prev; prev = current; current = next; } first = prev; } void display () { Node *temp = first; while (temp != NULL) { cout<<" "<<temp->data; temp = temp->next; } }; </pre>
--	---

Smart pointer implementation using c++

<pre> class RC { private: int count; // Reference count public: void AddRef() { </pre>	<pre> template < typename T > class SP { private: T* pData; // pointer RC* reference; // Reference count public: SP() : pData(0), reference(0) </pre>
---	--

```

    // Increment the reference count
    count++;
}

```

```

int Release()
{
    // Decrement the reference count and
    // return the reference count.
    return --count;
}
};

```

```

{
    // Create a new reference
    reference = new RC();
    // Increment the reference count
    reference->AddRef();
}

SP(T* pValue) : pData(pValue), reference(0)
{
    // Create a new reference
    reference = new RC();
    // Increment the reference count
    reference->AddRef();
}

SP(const SP<T>& sp) : pData(sp.pData),
reference(sp.reference)
{
    // Copy constructor
    // Copy the data and reference pointer
    // and increment the reference count
    reference->AddRef();
}

~SP()
{
    // Destructor
    // Decrement the reference count
    // if reference become zero delete the data
    if(reference->Release() == 0)
    {
        delete pData;
        delete reference;
    }
}

T& operator* ()
{
    return *pData;
}

T* operator-> ()
{
    return pData;
}

SP<T>& operator = (const SP<T>& sp)
{
    // Assignment operator
    if (this != &sp) // Avoid self assignment
    {
        // Decrement the old reference count

```

```

        // if reference become zero delete the old
data
        if(reference->Release() == 0)
        {
            delete pData;
+++++++ delete
reference;
        }

        // Copy the data and reference pointer
// and increment the reference count
pData = sp.pData;      reference =
sp.reference;
        reference->AddRef();
    }
    return *this;
}
};

```

```

void main()
{
    SP<PERSON> p(new Person("Scott", 25));  p-
>Display();
    {
        SP<PERSON> q = p;
        q->Display();
        // Destructor of q will be called here..

        SP<PERSON> r;
        r = p;      r-
>Display();
        // Destructor of r will be called here..
    }
    p->Display();
    // Destructor of p will be called here
    // and person pointer will be deleted
}

```

Managed C++

Managed C++ (C++/CLI)

1. Explain the difference between managed and unmanaged code?

MANAGED CODE is a code created by the .NET compiler. It does not depend on the architecture of the target machine because it is executed by the CLR (Common Language Runtime), and not by the operating system itself.

CLR and managed code offers developers few benefits, like garbage collection, type checking and exceptions handling.

UNMANAGED CODE is directly compiled to native machine code and depends on the architecture of the target machine. It is executed directly by the operating system. In the unmanaged code, the developer has to make sure he is dealing with memory usage and allocation (especially because of memory leaks), type safety and exceptions manually.

In .NET, Visual Basic and C# compiler creates managed code. To get unmanaged code, the application has to be written in C or C++.

COM components, ActiveX interfaces, and Windows API functions are examples of unmanaged code.

System.Runtime.InteropServices namespace provides a collection of classes useful for accessing COM objects, and native APIs from .NET.

2. How to access an exported function of a native DLL from .NET

The **DllImportAttribute** attribute provides the information needed to call a function exported from an unmanaged DLL.

```
[DllImport("vcwrap.dll")]
public static extern Int32 wcmo_init(...);
```

What is marshalling?

Marshalling is the process of creating a bridge between managed code and unmanaged code. In managed code, you have a String, while in the unmanaged world strings can be Unicode, non-Unicode, AnsiBstr etc.

MarshalAs attribute to specify what is the expected type on unmanaged side.

for example

```
[DllImport("somenativelibrary.dll")]
static extern int MethodA([MarshalAs(UnmanagedType.LPStr)] string parameter);
public static class Marshal Provides a collection of methods for allocating unmanaged memory, copying unmanaged memory blocks, and converting managed to unmanaged types, as well as other miscellaneous methods used when interacting with unmanaged code.
```

3. How to convert managed string to unmanaged char*

```
String^ name = L"Hello World";
```

```
IntPtr^ intptr = Marshal::StringToHGlobalAnsi(name); char
*str = (char*)(intptr->ToPointer());
```

```
public static IntPtr StringToHGlobalAnsi (string? s);
```

Marshal::StringToHGlobalAnsi(...) method to copy the Unicode string to unmanaged memory as an ANSI (one-byte) character. The method returns an **IntPtr** object that points to the beginning of the unmanaged string.

4. How to convert unmanaged char* to managed string

Marshal::PtrToStringAnsi(...) method Copies all characters up to the first null character from an unmanaged ANSI or UTF-8 string to a managed String, and widens each character to UTF-16.

```
char* name = "Hello World";  
IntPtr^ intptr = gcnew IntPtr((void *) name);  
String^ str = Marshal::PtrToStringAnsi(*intptr);
```

5. How to copy unmanaged array to managed array

```
public static void Copy (IntPtr source, int[] destination, int startIndex, int length);
```

```
int src_arr[] = {10,20,30}; // unmanaged array IntPtr^  
intptr = gcnew IntPtr((void *) src_arr); array<Int32>^  
targ_arr = gcnew array< Int32 >(3); Marshal::Copy  
(*intptr, targ_arr,0,3);
```

6. How to pass managed array to unmanaged array

Declares a pinning pointer, which is used only with the common language runtime.

A pinning pointer is an interior pointer that prevents the object pointed to from moving on the garbage-collected heap. That is, the value of a pinning pointer is not changed by the common language runtime.

This is required when you pass the address of a managed class to an unmanaged function so that the address will not change unexpectedly during resolution of the unmanaged function call.

```
array<double>^ arr = gcnew array<double>(5); pin_ptr<double>  
pinptr = &arr[0]; double *ptr = NULL; // Unmanaged pointer // void  
fun(double ptr[]) ptr = pinptr;
```

7. Explain about IntPtr

A platform-specific type that is used to represent a pointer or a handle. IntPtr size is 32 bits in 32-bit hardware, and 64 bits in 64-bit hardware.

for example

```
// to do //
```

<https://docs.microsoft.com/en-us/dotnet/api/system.intptr?view=net-5.0>

.NET Framework

1. Explain about garbage collection in .NET

Garbage collector is a feature of CLR which cleans unused managed (it does not clean unmanaged objects) objects and reclaims memory. It's a background thread which runs continuously and at specific intervals it checks if there are any unused objects.

What are generations in garbage collector (Gen 0, Gen 1 and Gen 2)

Gen 0: When application creates fresh objects, they are marked as Gen 0

Gen 1: When GC is not able to clear the objects from Gen 0 in first round it moves them to Gen 1 bucket

Gen 2: When GC visits Gen 1 objects and it is not able to clear them, it moves them Gen 2

Garbage collector will spend more time on Gen 0 objects rather than Gen 1 and Gen 2.

To enforce garbage collection in our code manually, you can run the command (C#)

System.GC.Collect();

2. What do the following acronyms in .NET stand for: IL, CIL, MSIL, CLI and JIT?

IL, or Intermediate Language, is a CPU independent partially compiled code. IL code will be compiled to native machine code using current environmental properties by Just-In-Time compiler (JIT).

JIT compiler translates the IL code to an assembly code and uses the CPU architecture of the target machine to execute a .NET application. In .NET, IL is called Common Intermediate Language (CIL), and in the early .NET days it was called Microsoft Intermediate Language (MSIL).

CLI, or Common Language Infrastructure, is an open specification developed by Microsoft. It is a compiled code library used for deployment, versioning, and security.

In .NET there are two CLI types: process assemblies (EXE) and library assemblies (DLL). CLI assemblies contain code in CIL, and as mentioned, during compilation of CLI programming languages, the source code is translated into CIL code rather than into platform or processor specific object code.

- When compiled, source code is first translated to IL (in .NET, that is CIL, and previously called MSIL).
- CIL is then assembled into a bytecode and a CLI assembly is created.
- Before code execution, CLI code is passed through the runtime's JIT compiler to generate native machine code.
- The computer's processor executes the native machine code.

3. Explain about static constructor?

A class can have static constructor. **Static constructors are called automatically, immediately before any static fields are accessed, and are generally used to initialize static class members.** It is called automatically before the first instance is created or any static members are referenced. Static constructors are called before instance constructors.

- we cannot use access modifiers on static constructor.
- static constructors cannot have parameters.

Give 2 scenarios where static constructors can be used?

- A typical use of static constructors is when the class is using a log file and the constructor is used to write entries to this file.
- Static constructors are also useful when creating wrapper classes for unmanaged code, when the constructor can call the **LoadLibrary ()** method

4. Explain about destructors, and finalize () in C#.Net

Destructors are invoked automatically, and cannot be called explicitly.

Destructors cannot be used with structs.

In class hierarchy, destructors will be called in reverse order from most derived to least derived.

For example, If I'm writing my code as...

```
class Abc
{
    ~Abc () { }
}
```

Here compiler will convert the class Abc code as...

```
class Abc
{
    protected override void Finalize ()
    {
        try
        {
            // ...
        }
        finally
        {
            base.Finalize();
        }
    }
}
```

Here the compiler-generated Finalize method contains the destructor body inside try block, followed by a finally block, that calls base class finalize. This ensures that destructors always call its base class destructor.

So, our conclusion from this finalize is another name for destructors in C#.

5. Explain about IDisposable interface (or) How to clean unmanaged objects of your class

If the application is using expensive external resource (unmanaged types), it is recommended to explicitly release the resource before the garbage collector runs and frees the object.

We can do this by implementing the Dispose method from the IDisposable interface that performs the necessary clean-up for the object. This can considerably improve the performance of the application.

Class Abc: IDisposable

```
{
    // Called from outside    public
    void Dispose ()
    {
        // user wants to release the resources.
        Dispose(true);

        // We have done the cleanup already, there is nothing left for
        // the finalizer(destructor) to do. So, lets tell the GC not to call it later.
        GC.SuppressFinalize(this);
    }
}
```

```

protected virtual void Dispose (bool isposing)
{
    if (isposing == true)
    {
        //someone wants the deterministic release of all resources.
        // let us release all the managed resource.
    }

    // release unmanaged resources, which will not be released by GC.
}

~Abc ()
{
    // the object went out of scope, and finalized is called.
    // let's call dispose in to release unmanaged resources.
    Dispose(false);
}
};

```

6. Explain the difference between the Stack and the Heap.?

Stack are stored value types (types inherited from `System.ValueType`), and in the Heap are stored reference types (types inherited from `System.Object`).

We can say the Stack is responsible for keeping track of what is actually executing and where each executing thread is (each thread has its own Stack).

The Heap, on the other hand, is responsible for keeping track of the data, or more precise objects.

7. Explain the difference between boxing and unboxing. Provide an example.

Boxing is the process of converting a value type to the type object, and unboxing is extracting the value type from the object. While the boxing is implicit, unboxing is explicit.

```

int i = 13;
object myObject = i;           // boxing
i = (int)myObject;             // unboxing

```

8. Discuss the difference between constants and read-only variables.

constants and read-only variable share many similarities, but there are some important differences:

- **Constants are evaluated at the compile-time**, while the **readonly variables are evaluated at the runtime**.
- **Constants** support only value-type variables, while **readonly variables** support value and reference type variables.
- **const fields are to be assigned at the time of declaration**, **readonly variables are declared as instance variable and assigned values in constructor**.
- Constants should be used when the value is not changing during the runtime, and read-only variables are used mostly when their actual value is unknown before the runtime.

9. Explain the differences between an Interface and an Abstract Class in .NET?

An interface merely declares a contract or a behaviour that implementing classes should have. It may declare only properties, methods, and events with no access modifiers. All the declared members must be implemented.

An abstract class provides a partial implementation for a functionality and some abstract/virtual members that must be implemented by the inheriting entities. It can declare fields too.

Neither interfaces nor abstract classes can be instantiated.

for example: -

Shape is an abstract base class, it is exposing **Color**, **Thickness** as data, **Draw ()**, and **Serialize ()** are the methods.

Draw () is defined as an abstract method, so we are hiding the implementation details from Shape class.

<pre>// Color, and Thickness fields will be shared. // Serialize common code will be shared public abstract class Shape { public int Color {get; set;} public int Thickness {get; set;} public virtual void Serialize (StreamWriter sw) {sw.Write ("{Color}, {Thickness},");} public abstract void Draw (); }</pre>	<pre>public class Line: Shape { public int Style {get; set;} public override void Serialize (StreamWriter sw) { base.Serialize (sw); sw.WriteLine ("{Style}"); } public override void Draw() { Console.WriteLine("Draws line.");}</pre>
--	---

Encapsulation is all about protecting the data, and Abstraction is hiding the implementation.

10. Explain about extension methods in C#

It is the ability to add new methods, to the existing class without inheritance.

for example: -

<pre>class Abc { public void fun1 () { } public void fun2 () { } }</pre>	<pre>static class Xyz { public static void fun3(this Abc a) { } public static void fun4(this Abc a, string str) { } } Here "this" keyword is the binding parameter.</pre>	<pre>Abc obj = new Abc (); obj1.fun1(); obj1.fun2(); obj1.fun3(); obj1.fun4("Hello");</pre>
--	---	---

- **Extension methods** are always defined as a static method, but when they are bound with any class or structure they will convert into non-static methods.
- When an extension method is defined with the same name and the signature of the existing method, then the compiler will print the existing method, not the extension method. Or in other words, **the extension method does not support method overriding.**
- You can also add new methods in the sealed class also using an extension method concept.
- It cannot apply to fields, properties, or events.

11. Difference between int.parse (...), Convert.ToInt32(...) and int.TryParse(...)

	<code>int.Parse (string s)</code>	<code>Convert.ToInt32(string s)</code>	<code>bool bReturn = int.TryParse (string s, out result)</code>
Integer in RANGE	returns integer value	returns integer value	result = int value bReturn = true
Null value	ArgumentNullException	returns "0"	result = 0 bReturn = false
Not in format	FormatException	FormatException	result = 0 bReturn = false
Value not in RANGE	OverflowException	OverflowException	result = 0 bReturn = false

12. Explain about delegate, and generic delegate?

A delegate is a type safe function pointer. A delegate type references to methods with a particular parameter list and return type.

A delegate instance associates with any method with a compatible signature and return type.

- The purpose of writing a delegate is to create a plugin, which can be pluggable into any object.
- Delegates allows us to create reusable objects in C#
- delegate is a reference type, which can be passed as an argument to any method.

A multicast delegate is a delegate that has references to more than one function. When you invoke a multicast delegate, all the functions the delegate is pointing to, are invoked.

Observer design pattern can be implemented by the multicast delegate. Observer design pattern also known as publisher/subscriber design pattern.

13. Explain about generics?

Generics are introduced in C#2.0. Generics allows us to design classes and methods decoupled from the data types. Generics are similar to the template functions, and template classes in C++

- Use generic types to maximize **code reuse, type safety, and performance.**
- The most common use of generics is to create collection classes.
- The .NET class library contains several generic collection classes in the **System.Collections.Generic** namespace.
- We can create our own **generic interfaces, classes, methods, events, and delegates.**

14. How to define a generic delegate?

```
public delegate T AddDelegate<T>(T arg1, T arg2);
public delegate T3 AddDelegate<in T1, in T2, out T3>(T1 arg1, T2 arg2);

class Program
{
    static void Main(string[] args)
    {
        AddDelegate<int> del1 = (x, y) => { return x + y; };
        AddDelegate<int, double, double> del2 = (x, y) => { return x + y; };
        Console.WriteLine(del1(10, 20));
        Console.WriteLine(del2(10, 1.234));
    }
}
```

15. Explain about Action, Func, Predicate delegates

Action delegate: - It takes 1 to 16 input parameters, and doesn't have return type.

Func delegate: - It takes 0 to 16 input parameters, and returns a value.

Predicate delegate: - It takes 1 input parameter, and returns true/false.

```

Action<int, int> del1 = (x, y) => Console.WriteLine("Addition is: {0}", x + y);
del1(10, 20);
Func<string, string, string> del2 = (str1, str2) => { return str1 + " " + str2; };
Console.WriteLine(del2("Shaik", "Nazeeruddin"));
Func<string, string> del3 = (str1) => "Mr." + str1;

Console.WriteLine(del3("Nazeeruddin"));

Predicate<string> del4 = (str1) => str1.Equals(str1.ToUpper());

string str = "Benefice";
Console.WriteLine("{0} is Uppercase: {1}", str, del4(str));

```

16. Explain Async, Await? `async` and `await` are the new keywords, which have been introduced in .NET framework 4.5 `async` and `await` are markers, which mark code positions from where the control should resume after the task(thread) completes.

Wherever you write “`await`”, that method must be marked with “`async`” keyword.

<pre> class Program { static void Main() { ProcessFile(); } static async void ProcessFile() { Task<int> task = new Task<int>(CountCharacters); task.Start(); Console.WriteLine("Processing file. Please wait..."); int count = await task; // wait until the long task finishes. Console.WriteLine(count.ToString() + " characters in file."); } } </pre>	<pre> static int CountCharacters () { // reading a big file and returns the count of characters Thread.Sleep(5000); return count; } </pre>
--	---

17. Explain Lazy loading with code?

Lazy loading is a concept where we delay the loading of the object until the point where we need it.

for example

A customer has a list of orders (which is loaded from database/file). While creating the object of customer, we are delaying the loading of orders. Whenever we need the orders, then only we will load the orders.

Advantages

- Minimizes start up time of the application.
- Application consumes less memory because of on-demand loading.
- Unnecessary database SQL execution is avoided.

<pre> public class Order { public int OrderNumber { get; set; } } public class Customer { public string CustomerName {get; set;} private Lazy<List<Order>> _orders = null; public List<Order> Orders { get { if (_orders == null) { _orders = new Lazy<List<Order>>() { () => LoadOrders(); } } return _orders.Value; } } } </pre>	<pre> private List<Order> LoadOrders() { List<Order> orders = new List<Order> (); orders.Add(new Order{OrderNumber = 1001}); orders.Add(new Order{OrderNumber = 1002}); orders.Add(new Order{OrderNumber = 1003}); return orders; } </pre>
---	---

18. Explain Dependency Injection?

<https://www.tutorialsteacher.com/ioc/lifetime-manager-in-unity-container>

Dependency injection allows program designs to be loosely coupled, and follow the **dependency inversion principle**. Dependency Injection, is a design pattern in which one or more dependencies (services) are injected into a dependent object (client).

Dependency Injection Involves 3 types of classes.

- **Client Class:** The client class (dependent class) is a class which depends on the service class
- **Service Class:** The service class (dependency) is a class that provides service to the client class.
- **Injector Class:** The injector class injects the service class object into the client class.

Injector class injects the service (dependency) to the client (dependent). The injector class injects dependencies in three ways: through a constructor, through a property, or through a method.

for example

CustomerService(dependency) object injected into the **CustomerBusinessLogic(dependent)** object by the **Injector (UI Application)**.

19. Explain about Unity Container, how it is used to resolve dependency objects, in dependency injection?

The Unity Container (Unity) is a **lightweight, extensible dependency injection container**. It's an open source IoC container for .NET applications supported by Microsoft.

Advantages: -

- Simplified type-mapping registration for interface type or base type.
- Supports registration of an existing instance.
- Supports code-based registration as well as design time registration.
- Automatically injects registered type at runtime through a constructor, a property or a method.

RegisterType<...>, **RegisterInstance<...>**, **Resolve<...>** methods will be called, on **UnityContainer** object.

20. Explain covariance and contravariance?

Covariance and contravariance introduced in C# 4.0

Covariance and contravariance enable implicit reference conversion for array types, delegate types, and generic type arguments.

Covariance preserves assignment compatibility and contravariance reverses it.

Covariance is upcasting in method return-type, and contravariance is down-casting in method-parameters.

We can say Covariance and Contravariance are the polymorphism extensions to array types, delegate types, and generic types.

Covariance deals with return type. For example, I have a delegate

```
public delegate Base MyDelegate (Derived d); // Derived class derived from Base
```

Here, I have a method

```
static Derived Method (Derived d)
{
    Console.WriteLine("Method");
    return new Derived ();
}
```

MyDelegate d = Method; // it allows, it is covariance

Another example

```
IEnumerable<Base> objs = new List<Derived> (); // it will throw exception in prior to C#4.0, and will work fine in C#4.0
```

Contravariance deals with parameters. For example, I have a delegate

```
public delegate Base MyDelegate (Derived d);
```

Here, I have a method

```
static Derived Method2(Base d)
{
    Console.WriteLine("Method");
    return new Derived ();
}
```

MyDelegate d = Method; // **contravariance, (params) Less Derived type assigned to More derived type**

21. Singleton Vs Static Class

- Singleton has an instance/object while static class is a bunch of static methods • Singleton can be extended e.g., through an interface while static class can't be.
- Singleton can be inherited which supports open/close principles in SOLID principles on the other hand static class can't be inherited and we need to make changes in itself.
- Singleton object can be passed to methods while static class as it does not have instance can't be passed as parameters

22. Difference between var and dynamic keyword?

Var	Dynamic
It is introduced in C# 3.0	It is introduced in C# 4.0
It is statically typed, and type of the variable decided at compile-time by the compiler	It is dynamically typed, and type of the variable decided at run-time by the compiler.
If it is not initialized, it will throw an error	If it is not initialized, it will not throw an error.
It supports IntelliSense in Visual Studio	It does not support IntelliSense in Visual Studio
It cannot be used for properties, returning values or parameters of the function. It can only be used as a local variable	It can be used for properties, return values or parameters of the function.

23. What is the difference between IEnumerable, and IQueryable?

IQueryable interface inherited from **IEnumerable**, so whatever **IEnumerable** can do, **IQueryable** can also do.

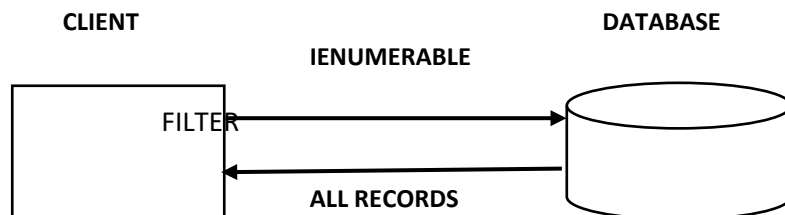
IQueryable interface is useful when your collection is loaded using LINQ or Entity Framework, and you want to apply filters on the collection.

For example, If I have an EntityFramework Object using IEnumerable

```
EmployeeContext context = new EmployeeContext ();

IEnumerable<Employee> employees = context.Employees.Where(emp => emp.Id == 2).ToList<Employee> ();
```

Here the “where” filter is executed on client side, here data is fetched from the database and then at the client, it scans and gets the record with employee Id is “2”.

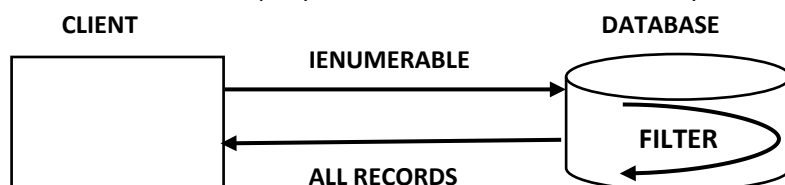


For example, If I have an EntityFramework Object using IQueryable

```
EmployeeContext context = new EmployeeContext ();

IQueryable<Employee> employees = context.Employees.Where(emp => emp.Id == 2).ToList<Employee> ();
```

Here the “where” filter is applied on the database using the “SQL” query., here the client is sending a request and, on the server-side a select query is filtered on the database and only necessary data is returned.



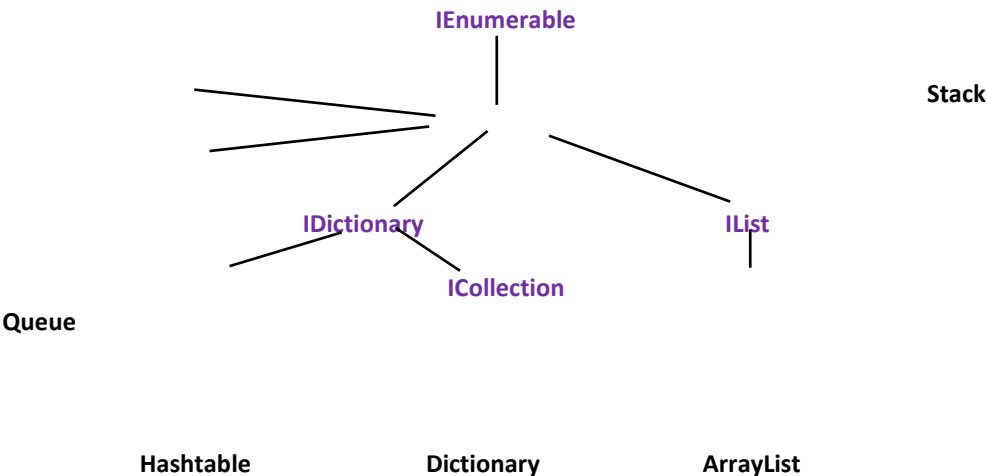
24. What is the difference between Dictionary and Hashtable?

Dictionary collection is a generic collection equivalent for Hashtable. Hashtable allows you to add Key/Value pair of any type (i.e., object). This leads to two problems, one is boxing and un-boxing issues, and second, it's not strongly typed.

25. What is the generic equivalent for ArrayList, Stack, Queue, and Hashtable?

Object based collections	Generic collections
ArrayList	List<int>
Stack	Stack<int>
Queue	Queue<int>
Hashtable	Dictionary<string, int>

26. Explain about IEnumerable, ICollection, IList, and IDictionary?



- IEnumerable:** - All collection classes use this interface and it helps to iterate through all the collection elements.
- ICollection:** - This interface has the count property and it is implemented by all collection elements.
- IList:** - this interface allows random access, insertion and deletion in to the collection. It is implemented by ArrayList.
- IDictionary:** - This interface is implemented by Hashtable and Dictionary.

27. Explain about LINQ

LINQ is an acronym for Language Integrated Query, and was introduced with Visual Studio 2008.

LINQ (Language Integrated Query) is uniform query syntax in C# and VB.NET to retrieve data from different sources and formats.

Supported data sources are: .NET Framework collections, SQL Server databases, ADO.NET Datasets, XML documents, and any collection of objects that support IEnumerable or the generic IEnumerable<T> interface.



There are two syntaxes of LINQ.

Lambda (Method) syntax

```
// data source
int[] arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// LINQ (LAMBDA EXTENSION METHOD)
var result = arr.Where(n => n % 2 == 0);
```

Query syntax

```
// data source
int[] arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// LINQ query
var result = from n in arr
              where n % 2 == 0
              select n;
```

Standard Query Operators in LINQ are actually extension methods for the **IEnumerable<T>** and **IQueryable<T>** types.

They are defined in the **System.Linq.Enumerable** and **System.Linq.Queryable** classes.

Standard Query Operators in Query Syntax

```
IEnumerable<Customer> result_customers = from customer in customers
                                         where customer.Age > 25
                                         select customer;
```

LINQ Query Operators

Standard Query Operators in Method Syntax

```
IList<Customer> res_customers = customers.Where(customer => customer.Age > 25).ToList<Customer>();
```

LINQ Extension Methods

Note:- Standard query operators in query syntax is converted into extension methods at compile time. So both are same.

<https://ankitsharmablogs.com/csharp-coding-questions-for-technical-interviews/>

Explain about Unit Testing, Integration Testing, Acceptance Testing, and Mock Testing

Multi-Threading

28. What is the difference between Process and Thread?

Any running program is called a **process**. Execution path within the process is called a **thread**. A process is a collection of resources like virtual address space, code, security contexts etc. A process can start multiple threads. Every process is start with a single thread called primary thread. You can create n number of threads in a process. Threads share the resources allocated to process.

A process is the parent and threads are his children.

29. Why we need multi-threading in our project?

Multi-threading is running the multiple threads simultaneously. Some main advantages are:

You can do multiple tasks simultaneously. for example, saving the details of user to a file while at the same time retrieving something from a web service.

Threads are much lightweight than process. They don't get their own resources. They used the resources allocated to a process.

Context-switch between threads takes less time than process

30. How to start a thread, and how to pass parameter to thread?

Create an instance of thread class, and call the **Start ()** method. Thread class constructor overloaded by taking argument either **ThreadStart** Delegate, or **ParameterizedThreadStart** Delegate.

By default, C# Threads are not background threads, if you are making them as background threads, you need call **Join ()** method, to make waiting of parent thread, until finishing of all child threads.

Thread class defined as...

```
class Thread
{
    // ThreadStart accepts: void ThreadProc ()
    public Thread (ThreadStart threadstart) {}

    // ParameterizedThreadStart accepts: void ThreadProc (object obj)
    public Thread (ParameterizedThreadStart parameterizedThreadStart) {}
}
```

How to start a thread?

How to pass arguments to thread?

<pre> using System; using System.Threading; namespace ConsoleApplication1 { class Program { static void Main(string[] args) { Thread t1 = new Thread (ThreadProc); t1.Start(); } private static void ThreadProc () { Console.WriteLine("Another Thread"); } } } </pre>	<pre> using System; using System.Threading; namespace ConsoleApplication1 { class Program { static void Main (string[] args) { Thread t1 = new Thread (ThreadProc); t1.Start("Hello"); } private static void ThreadProc (object args) { Console.WriteLine(args.ToString()); } } } </pre>
--	--

31. What is thread safety? Are .NET Framework methods being thread safe?

A code is called **thread-safe** if multiple threads can safely manipulate the shared data. In thread-safe code, a thread cannot modify the shared data at the same time when other thread is updating the data.

In .NET framework library all static methods are thread-safe and all the instance methods are not thread-safe.

What is race condition?

A race condition happens, when two or more threads want to update shared data at the same time.

What is volatile keyword?

- The **volatile keyword** indicates that a field might be modified by multiple threads that are executing at the same time.
- The compiler, the runtime system, and even hardware may rearrange reads and writes to memory locations for performance reasons.
- Fields that are declared volatile are not subject to these optimizations.

The volatile keyword can only be applied to fields of a class or struct. **Local variables cannot be declared volatile.**

The volatile keyword can be applied to fields of Reference types, Value Types, enum types, Pointer, and IntPtr

32. How can you share data between multiple threads?

There are two ways to share data between multiple threads.

- Concurrent Collections
 - ConcurrentQueue
 - ConcurrentStack
 - ConcurrentDictionary
 - ConcurrentBag

- Synchronization Primitives ○
 - Monitor** ○ **Mutex** ○ SpinLock
 - ReaderWriterLock ○
 - Semaphore** ○
 - AutoResetEvent** ○
 - ManualResetEvent** ○
 - InterLocked** ○
 - CountDownEvent ○ Barrier

33. What is synchronization and why it is important?

We use multiple threads to improve the performance of our application. **When multiple threads share data between there is a chance of data corruption.** When one thread is writing to the variable another thread is reading the same variable at the same time there is a chance of reading corrupt data.

34. What is deadlock?

A deadlock happens when two threads each wait for a resource held by the other, so neither can proceed.

Problem Scenario: -

1. I have two threads, Thread-1, and Thread-2
2. I have two mutex locks, Mutex-1, and Mutex-2
 - ✦ **Thread-1, and Thread-2 are in running state**, Thread-1 taken ownership on mutex-1, and Thread-2 taken ownership on mutex-2.
 - ✦ **Now Thread-1 trying to get ownership on Mutex-2**, and Thread-2 trying to get ownership on Mutex-1.

Solution: - We need to terminate one thread.

Popular advice to avoid deadlock is “Lock objects in a consistent order to avoid deadlocks”.

35. What is the difference between Thread and Task?

// todo

36. Difference between AutoResetEvent and ManualResetEvent?

// todo

37. Explain about semaphore

// todo

38. Explain about Mutex?

// to do

39. Difference between lock () and Monitor?

// to do.

40. Explain about ReaderWriterLockSlim?

Represents a lock that is used to manage access to a resource, **allowing multiple threads for reading or exclusive access (i.e., only one thread) for writing.**

```
ReaderWriterLockSlim cacheLock = new ReaderWriterLockSlim ();
```

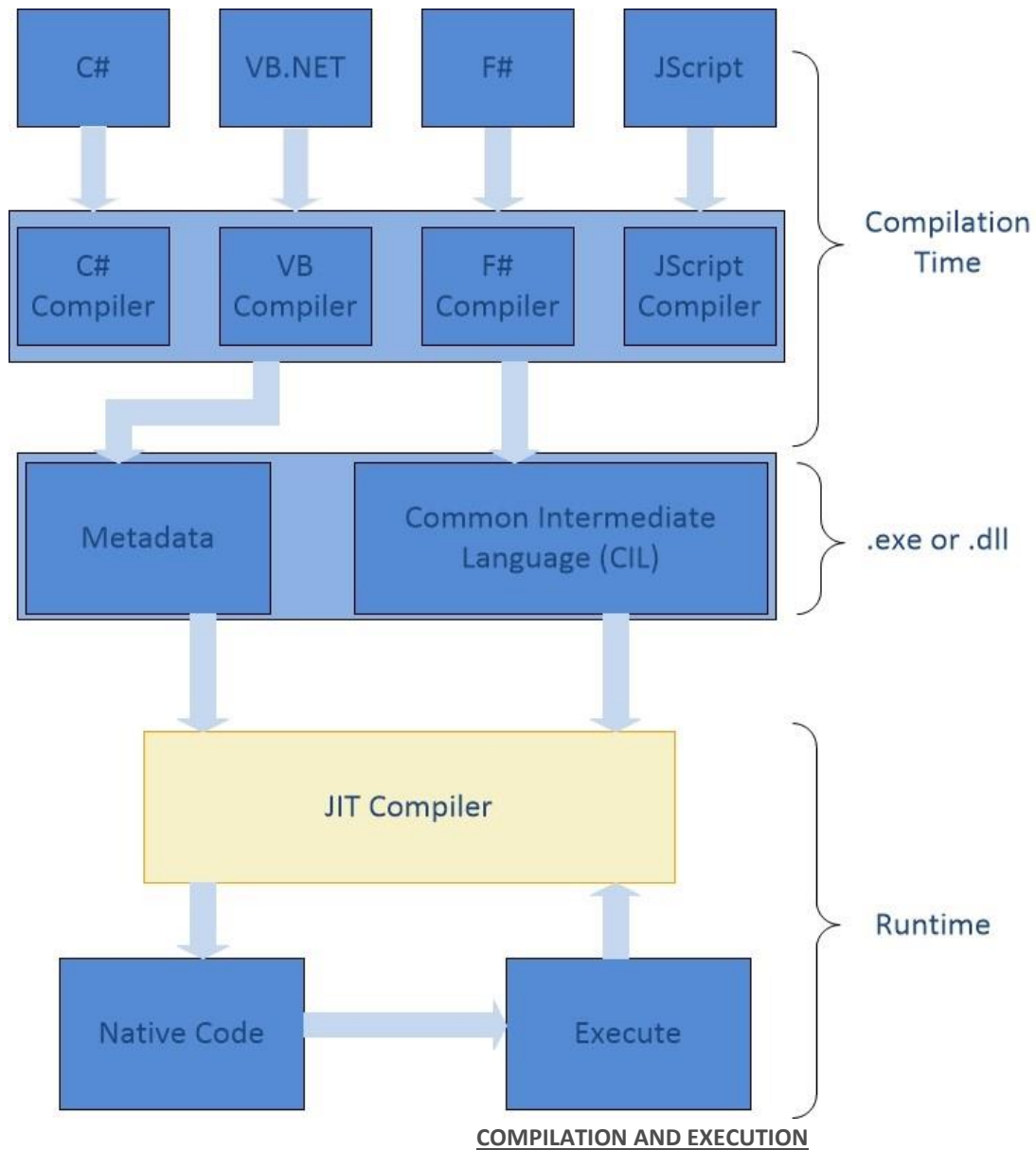
```
cacheLock.EnterReadLock();  
// read from resource  
cacheLock.ExitReadLock();
```

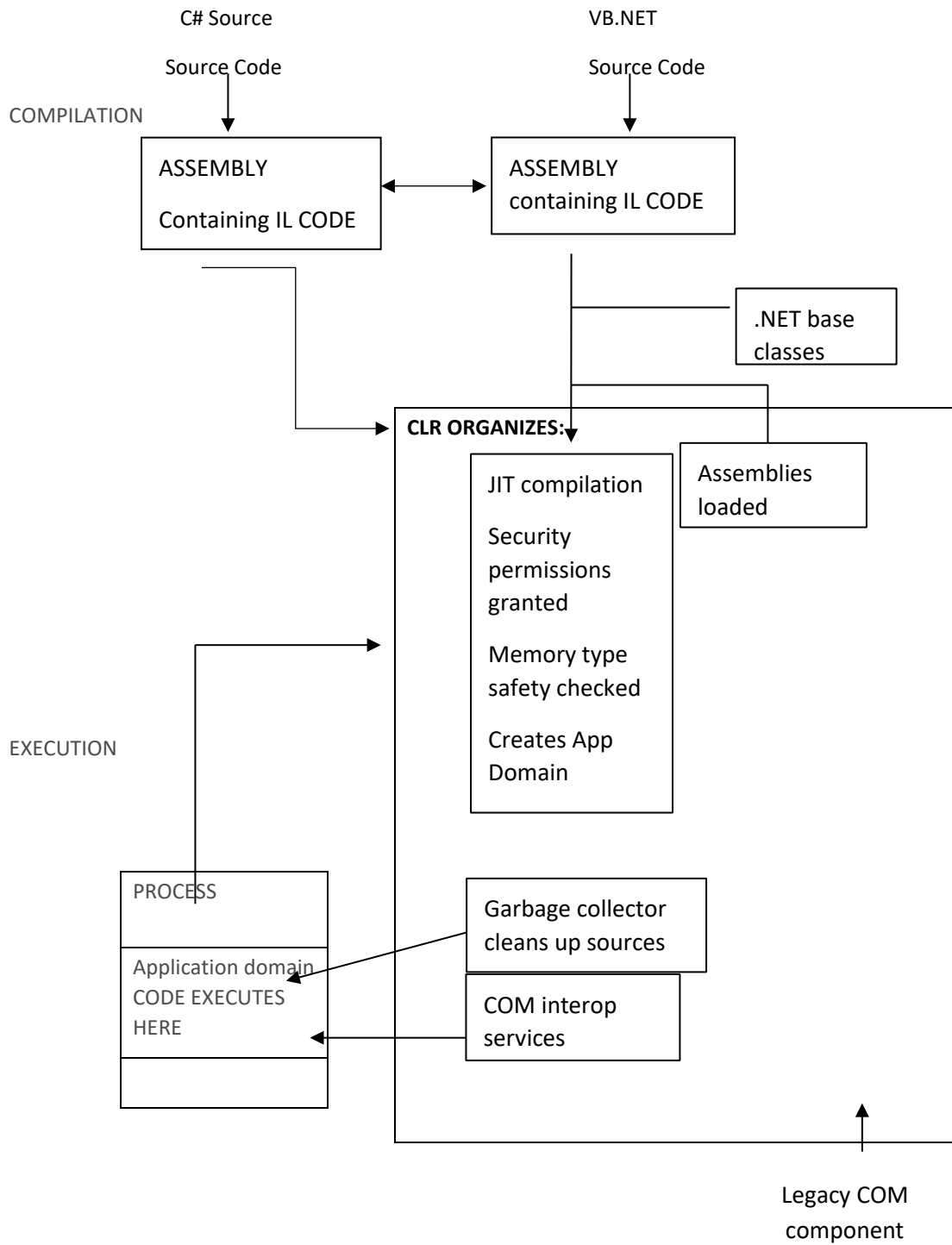
```
cacheLock.EnterWriteLock();  
// write to resource  
cacheLock.ExitWriteLock();
```

Coding Samples

Print Odd and Even Numbers using Task?

```
using System; using  
System.Threading; using  
System.Threading.Tasks;  
  
namespace OddVEvenTask  
{  
    class Program  
    {  
        static AutoResetEvent auto1 = new AutoResetEvent(false);  
static AutoResetEvent auto2 = new AutoResetEvent(false);  
  
        static void Main(string[] args)  
        {  
  
            var t1 = Task.Factory.StartNew(() => PrintOddNumbers());  
var t2 = Task.Factory.StartNew(() => PrintEvenNumbers());  
            Task.WaitAny(t1, t2);  
  
            Console.WriteLine("End");  
            Console.ReadLine();  
        }  
  
        static void PrintOddNumbers()  
        {  
            int[] arr = new int[] { 1, 3, 5, 7, 9, 11, 13, 15 };  
  
            foreach (var item in arr)  
{  
                Console.WriteLine(item);  
                auto1.WaitOne();  
                auto2.Set();  
            }  
        }  
  
        static void PrintEvenNumbers()  
        {  
            int[] arr = new int[] { 2, 4, 6, 8, 10, 12, 14 };  
  
            foreach (var item in arr)  
            {  
                auto2.WaitOne();  
                Console.WriteLine(item);  
                auto1.Set();  
            }  
        }  
    }  
}
```





MFC & WIN32 API

1. Explain about win32 program, WinMain ()

Win32 program execution starts with WinMain (...) int_stdcall

WinMain (HINSTANCE hInstance, // application instance handle

HINSTANCE hPrevInstance, // always null for 32 bit

LPSTR lpCmdLine, //command line string for the current instance.

int nCmdShow) // default value is 1(normal window), window display status

Define **WNDCLASSEX** Object

define WNDCLASSEX fields **lpszClassName, lpfnWndProc, hbrBackground...** Register WNDCLASSEX object by calling **RegisterClassEx (&obj)**

Call **CreateWindowEx (...)** // creates window

Call **ShowWindow (...)** by passing the window handle, which is returned by **CreateWindowEx ()**

Implement

message loop

MSG m;

while(**GetMessage(&m,0,0,0)** > 0)

```
{
    TranslateMessage
e(&m);
    DispatchMessage
(&m);
}
```

GetMessage (): - Reads messages from applications message queue and removes that message from queue. It is a blocking statement. It will be terminated, if it reads WM_QUIT message.

TranslateMessage (): - keyboard pressing will be converted as WM_CHAR message

DispatchMessage (): - dispatches the message to window procedure

Window procedure signature is...

LRESULT _stdcall WndProc (HWND hWnd, UINT MsgID, WPARAM wParam, LPARAM lParam)

HWND is window handle (message generated on which window) **UINT** is message id

WPARAM is 32-bit word parameter contains message additional information

LPARAM is 32-bit long parameter contains message additional information. Define a switch case in window procedure, which is processing the messages. If you are not processing any message, you can send that message details to **DefWindowProc (...)** to get the default behavior for your window.

2. What is a callback function?

A **callback** is a piece of executable code that is passed as an argument to other code, which is expected to call back (execute) the argument at some convenient time.



Win32 program, **WndProc (...)** is a **callback** function which is called by **Win32 API DispatchMessage (...)**

In my project we are defining **callback** functions (cashflows generation). Which are called by our **3rd party DLLs**.

for ex: - **icmo_cashflows** (icmop, ..., user_prepay_fcn, ...);

3. What is function pointer?

A **function pointer** is a variable that stores the **address of a function** that can later be called through that function pointer. **example:**

```
void fun (int x, int y) {...}
void (*pf) (int, int); pf =
&fun;
(*pf) (10, 20); // fun (10,20)
```

4. SendMessage (...) and PostMessage (...)?

PostMessage: Sends a message in the message queue associated with the thread and returns without waiting for the thread to process that message.

SendMessage: calls the window procedure for the specified window and does not return until the window procedure has processed the message.

PostMessage is an Asynchronous function, whereas **SendMessage** is a synchronous function.

5. PostThreadMessage (...)?

Posts a message to the message queue of the specified thread. It returns without waiting for the thread to process the message.

BOOL_stdcall PostThreadMessage (DWORD idThread, UINT Msg, _In_ WPARAM wParam, LPARAM lParam);

Messages sent by **PostThreadMessage** are not associated with a window.

There is a limit of 10,000 posted messages per message queue. To adjust this limit, modify the following registry key.

```

HKEY_LOCAL_MACHINE
SOFTWARE

    Microsoft

        Windows NT

            CurrentVersion
                Windows

                    USERPostMessageLimit

```

The minimum acceptable value is 4000.

6. GetMessage (...) and PeekMessage (...)?

Both are used to retrieve a message from application message queue.

- **GetMessage:**
 - a) It retrieves the message from the calling thread's message queue and removes from the message queue.
 - b) It blocks until it gets a message, or an error occurs.
 - c) It can filter message as per specified filter.
- **PeekMessage:**
 - a) It checks for the message in the calling thread's message queue and it retrieves the message if any present.
 - b) As specified in the argument, it may or may not remove the message from message queue.
 - c) It returns immediately if there is no message found in the message queue.

7. Difference b/w HINSTANCE and HWND?

HINSTANCE is an application instance handle, if you want to interact with your application from code, you must have the handle of your application.

HINSTANCE is provided by default with **WinMain (...)** and **DllMain (...)** as command line argument. Or you can get your current application instance handle by calling **GetModuleHandle (NULL)** API.

For sending messages to any window, you need a handle of that window **HWND**.

GetParent (...), function returns parent window handle, the parameter is child window handle. **GetWindow (...)** functions returns either parent window handle or child window handle, based on the second parameter specified.

EnumChildWindows (...), we can enumerate all child window handles.

8. How to create a dialog box in win32?

Dialog boxes are two types, **Modal and Modeless**.

After opening the modal dialog, we cannot access the parent window. After opening the modeless dialog, we can access the parent window.

Dialog box creation is to create the Dialog Resource using VS Resource editor and call the **DialogBox (...)** API by passing dialog resource id as argument, it creates the modal dialog. For modeless.... call **CreateDialog (...)** by passing dialog resource as argument and call **ShowWindow (...)** API.

9. What is message loop?

When user interacting with the window by clicking mouse or moving mouse or pressing any key on keyboard, each interaction details will be stored in the **MSG** object and it will be pushed to the application message queue by the OS.

WinMain (...) function having a while loop, by calling **GetMessage (...)**, **TranslateMessage (...)** and

DispatchMessage (...);

GetMessage (...) reads the message from application message queue.

TranslateMessage (...) is converting the keyboard strokes to WM_CHAR message.

DispatchMessage (...) is sending the message to the window procedure for processing.

10. WPARAM and LPARAM?

WPARAM and **LPARAM** are used to store additional message specific information.

Both are member variables of MSG structure.

if the message takes a pointer, the pointer is usually passed in the **LPARAM**, whereas if the

message takes a handle or an integer, then it is passed in the **WPARAM**.

If user presses any menu item or button on dialog box, menu item id and button id's are stored in

WPARAM. We can retrieve these values by using **LOWORD(wparam)** macro.

11. Explain about windows registry?

Windows registry is an operating system database, in which applications and system components, store and retrieve configuration data.

Prior to the Windows Registry, **.INI files** stored each **program's settings as a text file**, which did not provide user-specific settings in a multi-user scenario.

The **Windows Registry** is a hierarchical database, it stores information as **keys** and **values**. Registry keys are container objects similar to folders. Registry values are noncontainer objects similar to files.

Windows registry can be accessed through the command:

regedit There are seven predefined root keys.

HKEY_CLASSES_RO

OT

HKEY_CURRENT_US

ER

HKEY_LOCAL_MACH

INE HKEY_USERS

HKEY_CURRENT_CO

NFIG

12. How to store and retrieve data from windows registry? **RegCreateKeyEx (...)** // Create a key in windows registry **RegSetValue (...)** // Set a value to the existing key

RegDeleteKey (...) // Delete an existing key from registry

RegOpenKeyEx (...) //

RegQueryValueEx (...) // Retrieves value, associated with a key.

13. What is MFC?

Microsoft Foundation Class library is a **wrapper** around **Win32 API**. MFC is designed with the help of **OOPS concepts**. All **MFC** methods are calling internally **Win32 API functions**.

for ex: - **CFrameWnd :: Create (...)** method, internally calls **CreateWindowEX (...)** API

CPaintDC constructor calls **BeginPaint (...)** API to create **Device Context (DC)** object, **CPaintDC** destructor calls **EndPaint (...)** API.

14. Explain CObject?

CObject is the root base class for most of the Microsoft Foundation Class Library. **CObject** provides the following features to its sub-classes.

1. Diagnostic output support

You can write debug information to the visual studio debug output window. Derive a class from **CObject**. Override the **Dump (...)** method.

Void Dump (CDumpContext &dc) {...}

CDumpContext is a writing object, to write on debug output window.

2. Runtime class information support

Derive class from **CObject**.

Add **DECLARE_DYNAMIC(class_name)** macro in class declaration and

IMPLEMENT_DYNAMIC (class_name, base_class_name) macro in class implementation.

DECLARE_DYNAMIC macro is expanded as ...

Static const **CRuntimeClass *ptr;**

Static CRuntimeClass * GetThisClass ();

CRuntimeClass is a structure which is providing information about the class name, base class name, and object size and schema number.

So, you can get the **CRuntimeClass** object by calling **GetThisClass ()** static method.

3. Dynamic object creation support

Derive a class from **CObject**

Add **DECLARE_DYNCREATE(class_name)** macro in the class declaration and

IMPLEMENT_DYNCREATE (class_name, base_class_name) macro in the class implementation.

DECLARE_DYNCREATE macro expanded as...

DECLARE_DYNAMIC (class_name)

Static CObject * CreateObject ()

So, you can create object by invoking the **CreateObject ()** static method.

So, this macro is providing runtime class information and dynamic object creation support.

4. Serialization support

Derive a class from **CObject**

Add **DECLARE_SERIAL(class_name)** macro in the class declaration

And **IMPLEMENT_SERIAL(class_name, base_class_name, schema_number)** macro in the class implementation.

Override **Serialize (CArchive &)** method.

DECLARE_SERIAL macro expanded as... **DECLARE_DYNCREATE**
(class_name);

friend CArchive & operator>> (CArchive &, class_name* &);

Serialization is to store the state of the object to a persistent location.

CArchive is a kind of binary stream.

You can store the state (data) of the object to a persistent location using **CFile** object via CArchive object.

CArchive provides << and >> operators for writing and reading simple data types as well as

CObjects to and from a file.

CArchive is a bridge design pattern. You can connect CArchive object either CFile or CSocketFile.

15. CObject derived and not derived classes?

Most of the classes are derived from **CObject**. For ex: - **CCmdTarget, CWinThread, CWinApp, CWnd, CDialog** etc.,

Few classes are not derived from **CObject**. For ex: - **CArchive, CDumpContext, CPoint, CRuntimeClass, CRect, CSize** etc.,

16. Modal and modeless dialog boxes?

Modal	Modeless
Dialog box creation is the combination of Dialog template resource + CDialog derived class.	
Modal Dialog box created by invoking the CDialog constructor, by passing the dialog resource id as an argument, and DoModal () method of CDialog.	Modeless dialog box is created by calling CDialog's Create (...) method, and CDialog's ShowWindow () method.
Modal dialog box created on STACK	Modeless dialog box created on HEAP
After opening the modal dialog box, we cannot access the parent window, until it closed. For ex: - Open, Save, Font dialog boxes.	After opening the modeless dialog box, we can access the parent window. For ex: - Find and Replace dialog's.

Modal dialog box, CDialog::DoModal () is suspending the parent window message loop and starts its own message loop.	It doesn't have its own message loop, it depends on parent window message loop. Parent window forwards the messages to child window.
Modal dialog box is closed by calling EndDialog () , which hides the dialog box.	Modeless dialog box destroyed by calling DestroyWindow () , and post cleaning operations are handled by PostNcDestroy ()

17. Explain the process of creating a custom control ? (Or) What is MFC Sub-Classing?

MFC sub classing is to change the behavior or the visual appearance of the existing control. Derive an existing control class and handle messages to change the behavior or visual appearance.

For ex:- **Change the CEdit control class behavior to allow only alpha characters.**

Derive a class from CEdit. Handle WM_CHAR message.

Override **OnChar () handler**, filter each character and send alpha characters only to the base class CEdit, for processing.

Change the CStaticText or CEdit controls background and foreground colors by handling WM_CTLCOLOR message.

Override the OnCtlColor () method in CDialog Derived class.

Define a brush and return the brush handle to the System(Operating System)

Change the background color of CButton.

Derive a class from CButton.

Handle WM_DRAWITEM message

handler Override OnDrawItem () virtual function

Change the device context default foreground and background colors, which provided with DRAWITEMSTRUCT.

Note: Dialog box, button control property must be set to Owner Draw = TRUE.

Change the CListCtrl control list view items foreground and background colors Derive class from CListCtrl

Handle ON_NOTIFYREFLECT message Write

OnNMCustomDraw(NMHDR *) method.

Type cast NMHDR object to NMCUSTOMDRAW and LVCUSTOMDRAW objects respectively. And change the background and foreground colors with LVCUSTOMDRAW object.

18. explain about GDI objects?

Device Context Objects: CPaintDC, CClientDC, CMetafileDC,

CWindowDC. Brushes: CBrush (Solid Brush, Hatch Brush, Pattern Brush)

Pens: CPen , Font: CFont

19. What is message map? (or) MFC message map architecture? Windows applications are event driven programming model. Windows applications are responds by handling the messages.

- Message map architecture is implemented with **DECLARE_MESSAGE_MAP ()**, **BEGIN_MESSAGE_MAP ()**, and **END_MESSAGE_MAP ()** macros.
- We are adding **DECLARE_MESSAGE_MAP ()** macro in class declaration. **BEGIN_MESSAGE_MAP ()**, and **END_MESSAGE_MAP ()** macros in class implementation.
- We are inserting the message map macros in between **BEGIN_xxx_xxx** and **END_MESSAGE_MAP ()** macros.
for ex: - **ON_COMMAND ()**, **ON_BN_CLICKED ()** message map macros.
- **ON_COMMAND (msgid, functionaddress)** this is message map macro, it maps msg id with handler address. msgid and function address will be stored in the lookup table.
- Here **DECLARE_MESSAGE_MAP ()** macro is replaced with **_messageEntries []** array (which is lookup table, to store msgid and msghandler address).
- **_messageEntries []** array is created from **AFX_MSGMAP_ENTRY** structure, which has member variables as message id and function address.
- CWinApp base class CWinThread is implementing the message loop in **CWinThread::Run ()** function, message loop is reading the messages from message queue, and pumping those messages to the frame window.
- The pumped message id will be searched in the lookup table and appropriate message handler will be called.

20. Explain about win32 SDK Dll, MFC regular Dll and Extension Dll?

- **A Dll is a compiled binary code.**
- Dll is a shared file, which shares code among different applications/processes.
- Dll file has exported functions and global variables.

A DLL can be linked to any process in two ways.

- Implicit Linking
- Explicit Linking

Implicit Linking: - In implicit linking the **.LIB** file will be copied into the client's application/process. While loading client application **DllMainCRTStartup** loads the dll and allocates global variables memory in client process space. and calls **DllMain ()** function.

While deployment: - copy the **.dll**, **.lib** and **.h** file to the client project folder. Go to the project settings give the path of **.lib** file in project -> properties -> Linker -> Input -> Additional Dependencies: **abc.lib**

Explicit linking: - In explicit linking **.DLL** file will be enough, no need of **.LIB** file

.DLL file will be loaded at runtime of the application by calling...

HMODULE h = ::LoadLibrary (" dll name"); // it returns Module Handle

GetProcAddress (h, "function_name"); // it returns function address, call the function using function address.

FreeLibrary (h); // unloads the dll.

<u>MFC Regular DLL</u>	<u>MFC Extension DLL</u>
MFC Regular DLL's are - Statically linked to MFC Applications (or) - Dynamically linked to MFC Applications	MFC Extension DLL's are Dynamically linked to MFC Applications
Regular dll's can be linked to MFC or non-MFC Applications	Extension dll's can be linked with only MFC applications
Regular dll's are exporting global functions , and classes .	Extension dll's are exporting only classes .
Regular DLL entry point is CWinApp global object .	Extension DLL entry point is DllMain(...)

21. Process and thread?

Any running program is called process. Within the process the execution path is thread. A process will have at least one thread or can have more than one thread.

Process: A process is a collection of virtual memory space, code, data, and system resources. A thread is code that is to be serially executed within a process. A processor executes threads, not processes, so each 32-bit application has at least one process, and a process always has at least one thread of execution, known as the primary thread. A process can have multiple threads in addition to the primary thread.

Thread: A dispatchable unit of work (Executable path within the process). It is the thing that executes sequentially and is interruptible so that the processor can turn to another thread.

- **Race condition** - In a multithreaded application, a condition that occurs when multiple threads access a data item without coordination, possibly causing inconsistent results, depending on which thread reaches the data item first.

Deadlock - A **deadlock** happens when two threads each wait for a resource held by the other, so neither can proceed.

- **Concurrency** - The ability of more than one transaction or process to access the **same data at the same time**. For the data changes in the database table's cell as an example, this issue must be handled carefully.
- **Asynchronous call** - A call to a function that is executed separately so that the caller can continue processing instructions without waiting for the function to return.
- **Synchronous call** - A function call that does not allow further instructions in the calling process to be executed until the function returns.

22. What is dead lock?

A **deadlock** happens when two threads each wait for a resource held by the other, so neither can proceed.

Problem Scenario: -

1. I have two threads, Thread-1, and Thread-2
2. I have two mutex locks, Mutex-1, and Mutex-2

- Thread-1, and Thread-2 are in running state, Thread-1 taken ownership on mutex-1, and Thread-2 taken ownership on mutex-2.
- Now Thread-1 trying to get ownership on Mutex-2, and Thread-2 trying to get ownership on Mutex-1.

Solution: - We need to terminate one thread.

Popular advice to avoid deadlock is “Lock objects in a consistent order to avoid deadlocks”.

23. **_beginthread, _beginthreadex, CreateThread** differences?

CreateThread () is a raw Win32 API call for creating another thread of control at the kernel level.

_beginthread () & **_beginthreadex ()** are **C Runtime(CRT) Library** calls that call **CreateThread ()** behind the scenes.

- **beginthreadex** takes the 3 extra parameters to create thread. which are lacking in **beginthread ()**:
A) security descriptor for the new thread
B) initial thread state (running/asleep)
C) pointer to return ID of newly created thread
- The routine passed to **_beginthread ()** must be cdecl and has no return code, but the routine passed to **_beginthreadex ()** must be stdcall and returns a thread exit code.
- Handle that's returned by **beginthreadex** with the synchronization APIs, which you cannot do with **beginthread**. • **_endthread** automatically closes the thread handle, whereas **_endthreadex** does not.

24. **What is worker thread and UI thread?**

Worker Thread: - Worker threads don't have any message loop. Worker threads are used to run background tasks. Create worker thread by calling **AfxBeginThread (...)** by passing function address as parameter.

```
CWinThread* AfxBeginThread (AFX_THREADPROC pfnThreadProc, LPVOID pParam,
                             int nPriority = THREAD_PRIORITY_NORMAL,
                             UINT nStackSize = 0, DWORD dwCreateFlags = 0, LPSECURITY_ATTRIBUTES
                             lpSecurityAttrs = NULL
                             );
```

UI Thread: - User Interface(UI) thread will have the message loop. So, we can send messages to the UI Thread. Call **AfxBeginThread (...)** by passing Runtime Class pointer of CWinThread derived class object.

```
CWinThread* AfxBeginThread (CRuntimeClass* pThreadClass,
                             int nPriority = THREAD_PRIORITY_NORMAL,
                             UINT nStackSize = 0, DWORD dwCreateFlags = 0,
                             LPSECURITY_ATTRIBUTES lpSecurityAttrs = NULL
                             );
```

25. Explain about synchronization objects in MFC/Win32?

Synchronization is the act of coordinating concurrent actions for a predictable outcome.

Synchronization is particularly important when multiple threads access the same data;

types of synchronization objects Critical Section, Mutex, Event and Semaphore.

Difference between critical section and mutex?

Critical section and mutex both are providing exclusive lock on resource.

Critical Section: - can be used to synchronize the threads within the process. It's an application layer Object, it is a light weight object (**CRITICAL_SECTION** struct object created within The process).

Mutex: - Mutex is an **exclusive locking**, which allow just one thread to perform some activity or execute a section of code at a time. Their primary purpose is to let threads access writing state without interfering with one other.

- **Mutex** can be used to synchronize the threads within the process or across process boundaries.
- **Mutex** is a kernel level object.

API's used in critical section and mutex

Critical Section: -

CRITICAL_SECTION cs_obj; // creates critical section structure object, within the process
InitializeCriticalSection (&cs_obj); // initializes the critical section object, called by constructors.
DeleteCriticalSection (&cs_obj); // destroys the critical section object, called by destructors.

EnterCriticalSection (&cs_obj); // creates lock

LeaveCriticalSection (&cs_obj); // releases lock

Mutex: -

HANDLE hMutex = CreateMutex (...); // creates named/un-named mutex object, with/without ownership of calling threads.

HANDLE hMutex = OpenMutex (...); // reads handle of named mutex object.

WaitForSingleObject (hMutex, INFINITE); // waiting for ownership on lock

ReleaseMutex (hMutex); // release ownership on lock

CloseHandle (hMutex); // closes the mutex object

How critical section and mutex used in your project: -

- Thread-Safe singleton class (configuration settings class) is using critical section within the process.
- One log file is logged by multiple processes, here exclusive lock is named mutex object.
- While generating the reports, common data objects are locked by critical section within the process.

various waiting results are: **WAIT_OBJECT_0, WAIT_TIMEOUT, WAIT_ABANDONED**

What is Event object?

Event: - Event object allow a thread to block until receiving a notification from other thread(s). The signaling constructs include **Manual Reset Event** and **Auto Reset Event**.

An event object is a synchronization object whose state can be explicitly set to signal by use of the

SetEvent or PulseEvent function.

API's used in Event Object

HANDLE hEvent = CreateEvent (...); // creates manual/auto reset event, with initial state signaled/non-signaled. If second parameter is FALSE, it is auto-reset event

HANDLE hEvent = OpenEvent (...); // reads existing event object handle, with a specified name.

SetEvent (...); // explicitly makes signaled state

ResetEvent (...); // explicitly makes non-signaled state.

Manual-reset-event: - An event object whose state remains signaled until it is explicitly reset to non-signaled by the **ResetEvent** function.

Auto-reset-event: - An event object whose state remains signaled until a single waiting thread is released. No need of calling **ResetEvent** function, to make non-signaled state of the event object.

Semaphore: - A resource can be shared by more than two threads, using binary semaphore. For example, a file shared with multiple threads with reading access.

API's used in Semaphore Object

HANDLE hSemaphore = CreateSemaphore (...); // create the semaphore and to specify initial and maximum counts (No. of threads can access).

WaitForSingleObject (hSemaphore, INFINITE); // determine whether the semaphore's current count permits the thread to get lock on resource.

ReleaseSemaphore (...); // increments the semaphores count, thus enabling another waiting thread to get access on shared resource.

26. Explain about Inter-Process Communication(IPC)?

Inter-Process Communication (IPC) is a set of techniques for the exchange of data among multiple threads in one or more processes. Processes may be running on one or more computers connected by a network.

IPC techniques include **Named Pipes, Winsock, File Mapping, Mailslot, Remote Procedure Calls (RPC)**, etc.

Named Pipe: - Named pipes is a mechanism for **one-way or bi-directional inter-process communication between a pipe server and one or more pipe clients** in the local machine or across computers in an intranet.

Named Pipe	
Serve r	Clie n t
Create a named pipe. (CreateNamedPipe (...))	Try to open a named pipe. (CreateFile (...))
Wait for the client to connect. (ConnectNamedPipe (...))	Set the read mode and the blocking mode of the specified named pipe. (SetNamedPipeHandleState (...))
Read client requests from the pipe and write the response. (ReadFile (...) , WriteFile (...))	Send a message to the pipe server and receive its response. (WriteFile (...) , ReadFile (...))
Disconnect the pipe and close the handle. (DisconnectNamedPipe (...) , CloseHandle (...))	Close the pipe. (CloseHandle (...))

Source: <https://www.codeproject.com/Articles/13724/Windows-IPC>

27. How can you create a socket?

The steps involved in establishing a socket on the client side are as follows:

- Create a socket with the **socket ()** system call
- Connect the socket to the address of the server using the **connect ()** system call
- Send and receive data. There are a number of ways to do this, but the simplest is to use the **read ()** and **write ()** system calls.

The steps involved in establishing a socket on the server side are as follows:

- Create a socket with the **socket ()** system call
- Bind the socket to an address using the **bind ()** system call. For a server socket on the Internet, an address consists of a port number on the host machine.
- Listen for connections with the **listen ()** system call
- Accept a connection with the **accept ()** system call. This call typically blocks until a client connects with the server.
- Send and receive data

28. Explain about server socket/client socket?

... To do ...

<https://www.geeksforgeeks.org/socket-programming-cc/>

29. How to handle multiple clients on SERVER, without multi-threading?

... To do ...

<https://www.geeksforgeeks.org/socket-programming-in-cc-handling-multiple-clients-onserver-without-multi-threading/>

30.Explain about Doc/View architecture?

Separating **data** and its **representation/rendering** in between two different objects is called

Document/View architecture application.

Using Document/View Architecture, we can create two kinds of applications.

- **Single Document Interface(SDI)** and
- **Multiple Document Interface(MDI)**

An SDI application is created with 4 Derived classes

CDocument derived

CView derived

CFrameWnd derived

CWinApp derived

SDI, Will be created by constructing **CSingleDocTemplate** object, by passing the **Resource ID**, **CDocument**, **CFrameWnd** and **CView RuntimeClass Pointers**.

MDI application is created with 5 Derived classes

CDocument derived

CView derived

CFrameWnd (or) **CMDIFrameWnd** derived

CMDIChildWnd derived (It creates child frame windows)

CWinApp derived

CDocument object holds the data

CView display's the data.

CFrameWnd Routes the messages to active view and view associated document object.

CWinApp is the application class, which is responsible for managing the **CDocTemplate** Objects and, reading the messages from message queue, and pumping those messages to The frame window.

MDI, will be created by constructing **CMultiDocTemplate** object by passing **Resource id**, **Document**, **ChildFrame** and **View RuntimeClass Pointers**.

Doc/View is an **observer** design pattern. (Here you have to explain **observer design pattern**)

CDocument and **CView** objects are binded together with **CDocTemplate** object.

If document is updated all of its views also will be updated.

31. Difference between SDI and MDI, and explain the creation of both?

Single Document Interface (SDI): - We can open only one document at a time. For ex: - Notepad.

Multiple Document Interface (MDI): - We can open more than one document at a time. For ex: - MS. Word 95/98.

Either SDI or MDI application, a document object can be attached with multiple views.

How to display multiple views for SDI application?

Create **CSplitterWnd** object with one row and two columns. Here each column, one view will be displayed.

For SDI application, to create more than one view, we have to handle the **CFrameWnd::OnCreateClient (...)** message handler.

```
BOOL myframe::OnCreateClient ( LPCREATESTRUCT l, CCreateContext *p )
{
    splitter.CreateStatic ( this, 1, 2 ) ;
    splitter.CreateView ( 0, 0, RUNTIME_CLASS ( graphicsview ),
                        CSize ( 128, 0 ), p ) ;
    splitter.CreateView ( 0, 1, RUNTIME_CLASS ( textview ),
                        CSize ( 0, 0 ), p ) ;
```

32. Command routing in SDI?

CWinThread::Run (...) function will read's messages from application message queue and pumps those messages to the frame window.

CCmdTarget::OnCmdMsg (...) method defines the **Command Routing** in MFC. For Doc/View architecture applications **CFrameWnd::OnCmdMsg (...)** implements the **command routing**.

CFrameWnd::OnCmdMsg (...) will route the messages to View and document.

CFrameWnd: OnCmdMsg (...) pumps the message through active view.

- **CFrameWnd::OnCmdMsg (...)** will call **GetActiveView ()**, which return's active view pointer.
- With view pointer it calls **GetDocument ()**, which returns document pointer. So, with doc pointer message will be sent to document object.
- If message is not handled by document, then it will be handled by the view class.

If message is not handled by view or document then, **CFrameWnd::OnCmdMsg (...)** pumps the message through frame window.

If message is not handled by frame window then, **CFrameWnd::OnCmdMsg (...)** pumps the message through Application class.

If message is not handled by application class then, **CFrameWnd::OnCmdMsg (...)** pumps the message to the **DefWindowProc (...)**

How to update the document object from view?

By calling **GetDocument ()**, we will get document pointer. So, we can update document members.

How to update all views from document object?

UpdateAllViews () will update all views. (or) from **CDocument** derived class, we can call **GetFirstView ()** and **GetNextView ()** functions, so we can get **CView** pointer and we can call **OnUpdate ()** on **CView** pointer.

33. Working with WM_TIMER in MFC

We create a timer, specify its **elapse time**, and (optionally) attach it to a window. After the timer is created, it sends **WM_TIMER** messages to the **window message queue**, or if no window was specified, to the **application queue**. We can process this message to call the code that we want to be **executed in regular time intervals**. The timer will send **WM_TIMER** messages until it is destroyed.

To create a timer, we will use a Win32 function:

`UINT_PTR SetTimer (HWND hWnd, UINT_PTR timer_identifier, UINT elapsed_time, TIMERPROC timer_callback_fcn);` MFC function:

`UINT CWnd::SetTimer (UINT_PTR timer_identifier, UINT elapsed_time, void (CALLBACK EXPORT *lpfnTimer) (HWND, UINT, UINT_PTR, DWORD));`

To Kill or destroy the timer

`BOOL KillTimer (HWND hWnd, UINT_PTR timer_identifier);` // Win32 API

`BOOL CWnd::KillTimer (UINT_PTR timer_identifier);` // MFC

34. What is the difference between debug build and release build?

- The **Debug configuration of your program is compiled with full symbolic debug information and no optimization**. Optimization complicates debugging, because the relationship between source code and generated instructions is more complex.
- The **Release configuration of your program contains no symbolic debug information and is fully optimized**. Debug information can be generated in PDB Files, depending on the compiler options that are used. Creating PDB files can be very useful if you later have to debug your release version.

35. How to avoid release mode issues? Your application is working fine in debug, and failed in RELEASE?

- Always **regularly test both the Debug and Release mode versions** of your applications frequently.
- **Don't put your release build needed code in ASSERT, and TRACE macros**. ASSERT and TRACE macros will be removed in RELEASE build. Change ASSERT macros to VERIFY
- Look for any code you've placed inside of a **"#ifdef _DEBUG" wrapper that might be needed in the Release version**.

- Never assume that your variables are initialized unless you've explicitly initialized them in your code. This includes global variables, automatic variables, malloced data, and newed data.
 - **Variables and allocated memory are often initialized differently in Debug versions than in Release version of software.**
- If you have **references to resources that have been removed from your resource file**, your debug version may work but your release version may crash.
- When you start a project **turn the warning level to "Level 3" or "Level 4"** and make sure your code compiles with no warnings before you check your code in.
 - **Compile warning level - 4, may expose initialization problems.**

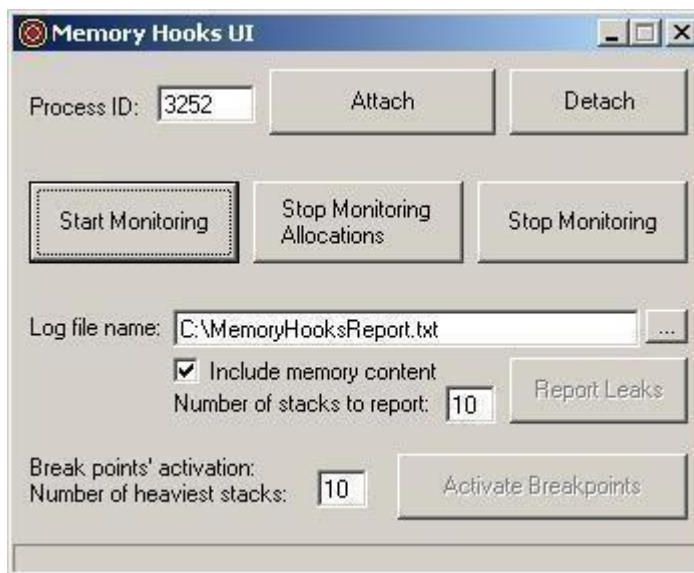
36. How to debug a release build?

It is possible to debug in Release mode. The first step is to turn on symbols:

- In Project Settings (Alt-F2) under the **"C++/C tab"** set the category to **"General"** and change the **Debug Info** setting to **"Program Database"**.
- Under the **"Link tab"** check the **"Generate Debug Info"** tab.
- **Recompile using "Rebuild All"**

37. How to find memory leaks, best practices to prevent memory leak? /// TO DO ///

Windows Leaks Detector



38. What are the design patterns implemented in MFC Framework?

Creational: Singleton Pattern

First step in any MFC application is the creation of application object (object of class derived from CWinApp). There should be only one application object in an instance of MFC application. CWinApp is designed to make sure that only application object is present in a given instance.

CWinApp and its descendants are called Singleton Classes. A class (CWinApp or its descendant) that assures a maximum of ONE object of its type at a given time and provides a global access point (AfxGetApp() method) to this object is a Singleton class.

As this principle is applied over and over again to solve recurring object "creational" problems, this becomes a pattern. Singleton Pattern ensures that a class only has one instance and provides a global access point to it. The article Creating Singleton Objects using Visual C++ talks about different approaches for implementing Singletons.

Structural: Bridge Pattern

Bridge Pattern is all about decoupling an abstraction (interface) from its implementation so that the two can vary independently. In MFC, the process of storing/retrieving an object to/from a persistence mechanism (like a file) is called Serialization. MFC uses the Bridge Pattern to implement Serialization. CArchive and CFile classes implement object Serialization. CArchive class provides the interface for writing/reading an object to/from a persistence mechanism whereas the CFile and its sub classes provides implementation for different persistence mechanisms such as memory, disk file, sockets etc.

A CArchive object is configured with an object of class CFile (or a derived class) during its construction, from which it obtains the necessary information for serialization, including the filename and type of the requested operation (a read or write). Client performing the Serialization operation can use CArchive object without regarding the persistence mechanism implemented by CFile classes.

Bridge Pattern - Bridging the gap between Interface and Implementation talks about Bridge pattern in detail.

Behavioral: Observer Pattern

The Observer Pattern is intended to "Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically". An object that is subjected to change is called a Subject and an object that depends on the Subject's state is called an Observer.

MFC uses a Document/View variant of the Observer Pattern. MFC's famous Document/View architecture uses this variant. A document contains the data object and acts as a Subject. A view is a window object through which the user updates the document and it acts as an Observer. A document can have multiple views. Whenever the data in the document is changed by one of the views, it updates the document by calling UpdateAllViews method, with optional hint about the modification. To inform about the change to other views, the document object calls OnUpdate method for each view attached to it (except the view that called UpdateAllViews).

Derived view classes can override the OnUpdate method and update themselves by querying the data from the document.

CDocument::AddView (CView *pView) // attaches a view to the document

CDocument::RemoveView (CView *pview) // detaches a view from the document

CDocument::UpdateAllViews (...) // Notifies all views that document has been modified.

CView::OnUpdate () // Called to notify a view, that its document has been modified.

39. Explain various time complexities of ALGORITHMS & Data Structures.

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$		$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Stack	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$		$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Queue	$\Theta(n)$	$\Theta(n)$			$\Theta(n)$	$\Theta(n)$			$\Theta(n)$
Singly-Linked List			$\Theta(1)$				$\Theta(1)$	$\Theta(1)$	$\Theta(n)$

Doubly-Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$						$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	
	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$						$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	
Hash Table	N/A	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$						N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary Search Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$						$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B-Tree	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$						$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
	$\Theta(\log(n))$													$O(n)$

Red-Black Tree	(n)	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$		$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	
Quick Sort										

COM

1.What is COM?

The component object model is an object based programming model designed to provide software interoperability, that is to allow two or more applications or "components" to easily cooperate with one another, even if they were written by different vendors at different times, in different programming languages, or if they are running on different machines running different operating systems.

To support interoperability features, COM defines and implements mechanisms that allow applications to connect to each other as software objects.

Client:

client never has direct access to the COM object in its entirety. Instead client always access the object through clearly defined contracts; the interfaces that the object supports, and only those interfaces.

2.What is interface?

A class can be instantiated to form an object.

An interface cannot be instantiated by itself because it carries no implementation.

An interface is just a related group of functions and is the binary standard through which clients and objects communicate.

Interfaces are strongly typed: Every interface has its own interface identifier (a GUID) thereby eliminating any chance of duplication. So, each interface must be assigned with one GUID.

Interfaces are immutable: Interfaces are never versioned, so it avoids versioning problems.

A new version of an interface, created by adding or removing functions or changing the semantics, is an entirely new interface and assigned with new GUID.

COM Server Types:-

In-Proc Server (COM DLL): A server that can be loaded into the client's process space and serves "in-process objects.

Out-Proc Server (COM EXE):A server that runs in a separate process on the same machine as the client and serves "local objects."

Remote Server (COM DLL/Exe): A server that runs on a separate machine and therefore always runs in another process as well to serve "remote objects." Remote servers may be implemented in either DLLs or EXEs; if a remote server is implemented in a DLL, a surrogate process will be created for it on the remotemachine.

COM SERVER

To Implement a com server

Define Interfaces which are IUnknown derived.

IUnknown interface has three methods, AddRef(), QueryInterface(), and Release(). Here AddRef() and Release() maintains the lifecycle of the component object, and QueryInterface returns the requested interface pointer to the client.

Define COM class which implements IUnknown compliance (IUnknown derived) interfaces.

Define a factory class which implements IClassFactory.

Factory class is responsible for creating component object.

Why because client is not provided to access the component object directly.

Define DllGetClassObject, which creates object for ClassFactory and returns IClassFactory pointer to the client.

Define DllRegisterServer and DllUnregisterServer these are registering or un registering the COM Dll entities in system registry.

COM CLIENT

CoInitialize(NULL) will initialize the COM Libraries

CoGetClassObject(...) it calls DllGetClassObject on com server and returns iclassfactory pointer.

by using iclass_factory pointer call CreateInstance(...). It creates component object and returns IUnknown interface pointer to the client.

by using IUnknown interface pointer call QueryInterface by requesting the required interface pointer.

QueryInterface will return the requested interface pointer.

call methods which are associated with your required interface. call

release after using that interface

If reference interface count become zero. means client is not using any interface then it(server) will call destructor on component class.;

CoInitialize(NULL) will initialize the COM Libraries

Call CoCreateInstance(...) by requesting the required interface pointer.

On server side CoCreateInstance() internally calls, CoGetClassObject and DllGetClassObject and gets the classfactory pointer and calls CreateInstance by requesting the interface pointer, which passed in CoCreateInstance(..).

QueryInterface will return the requested interface pointer.

call methods which are associated with your required interface. call

release after using that interface

If reference interface count become zero. means client is not using any interface then it will call destructor on component class.

3.Difference between Normal DLL and COM DLL?

Normal DLL	COM DLL
1. Exports global functions	1. Exports interfaces
2. No Registration	2. Registered in windows registry
3. No versioning	3. Versioning is possible
4. We can modify the function signatures	4. COM Rules are not supported to modify the function signatures. HERE interfaces are immutable.
5. DLL will be unloaded whenever No Exe is using, that DLL.	5. com OBJECT will be destroyed if, interface count becomes ZERO.

4.What is IUnknown? What methods are provided by IUnknown?

It is a generally good idea to have an answer for this question if you claim you know COM in your resume. Otherwise, you may consider your interview failed at this point. IUnknown is the base interface of COM. All other interfaces must derive directly or indirectly from IUnknown. There are three methods in that interface: AddRef, Release and QueryInterface.

5.What are the purposes of AddRef, Release and QueryInterface functions?

AddRef increments reference count of the object, Release decrements reference counter of the object and QueryInterface obtains a pointer to the requested interface.

What should QueryInterface functions do if requested object was not found?

Return E_NOINTERFACE **and** nullify its out parameter.

6.How can would you create an instance of the object in COM?

Well, it all depends on your project. Start your answer from CoCreateInstance or CoCreateInstanceEx, explain the difference between them. If interviewer is still not satisfied, you'll have to explain the whole kitchen behind the scenes, including a difference between local server and inproc server, meaning and mechanism of class factory, etc. You may also mention other methods of object creation like CoGetInstanceFromFile, but discussion will likely turn to discussion of monikers then.

7.What happens when client calls CoCreateInstance?

Again, all depends on the level of detail and expertise of interviewer. Start with simple explanation of class object and class factory mechanism. Further details would depend on a specific situation.

8.What the limitations of CoCreateInstance?

Well, the major problems with CoCreateInstance is that it is only able to create one object and only on local system. To create a remote object or to get several objects, based on single CLSID, at the same time, one should use CoCreateInstanceEx.

9.What is aggregation? How can we get an interface of the aggregated object?

Aggregation is the reuse mechanism, in which the outer object exposes interfaces from the inner object as if they were implemented on the outer object itself. This is useful when the outer object would always delegate every call to one of its interfaces to the same interface in the inner object. Aggregation is actually a specialized case of containment/delegation, and is available as a convenience to avoid extra implementation overhead in the outer object in these cases. We can get a pointer to the inner interface, calling QueryInterface of the outer object with IID of the inner interface.

10.C is aggregated by B, which in turn aggregated by A. Our client requested C. What will happen?

QueryInterface to A will delegate request to B which, in turn, will delegate request for the interface to C. This pointer will be returned to the client.

11.What is a moniker ?

An object that implements the IMoniker interface. A moniker acts as a name that uniquely identifies a COM object. In the same way that a path identifies a file in the file system, a moniker identifies a COM object in the directory namespace.

What's the difference, if any, between OLE and COM?

OLE is build on top of COM. The question is not strict, because OLE was built over COM for years, while COM as a technology was presented by Microsoft a few years ago. You may mention also that COM is a specification, while OLE is a particular implementation of this specification, which in today's world is not exactly true as well, because what people call COM today is likely implementation of COM spec by Microsoft.

What's the difference between COM and DCOM?

Again, the question does not require strict answer. Any DCOM object is yet a COM object (DCOM **extends** COM) and any COM object may participate in DCOM transactions. DCOM introduced several improvements/optimizations for distributed environment, such as MULTI_QI (multiple QueryInterface()), security contexts etc. DCOM demonstrated importance of surrogate process (you cannot run in-proc server on a remote machine. You need a surrogate process to do that.) DCOM introduced a load balancing.

What is a dual interface?

Dual interface is one that supports both - IDispatch interface and vtbl-based interface. Therefore, it might be used in scripting environment like VBScript and yet to use power and speed of vtbl-based interface for non-scripting environment. Discussion then may easily transform into analyzing of dual interface problems - be prepared to this twist.

Can you have two dual interfaces in one class?

Yes. You **may** have two dual interfaces in one class, but only one of them may be default. The bottom line is that you cannot work with two dual interfaces at the same time due to nature of dual interface! To support two dual interfaces in VB you would write something like:

```
dim d1 as IDualInterface1
dim d2 as IDualInterface2

set d1 = new MyClassWithTwoDuals
set d2 = d1
```

In ATL's class you would have to use macro COM_INTERFACE_ENTRY2(IDispatch, IDualInterface1), to distinguish between different dual interfaces.

What is marshalling by value?

Some objects can essentially be considered static: regardless of which methods are called, the state of the object does not change. Instead of accessing such an object remotely, it is possible to copy the static state of the object and create a new object with the same state

information on the caller side. The caller won't be able to notice the difference, but calls will be more efficient because they do not involve network round trips. This is called "marshaling by value".

What is a multi-threaded apartment (MTA)? Single-threaded apartment (STA)?

This is pretty difficult question to describe shortly. Anyway, apartments were introduced by Microsoft in NT 3.51 and late Windows 95 to isolate the problem of running legacy non-thread safe code into multithreaded environment. Each thread was "encapsulated" into so called single-threaded apartment. The reason to create an object in apartment is thread-

safety. COM is responsible synchronize access to the object even if the object inside of the apartment is not thread-safe. Multithreaded apartments (MTA, or free threading apartment) were introduced in NT 4.0. Idea behind MTA is that COM is not responsible to synchronize object calls between threads. In MTA the developer is responsible for that. See "Professional DCOM Programming" of Dr. Grimes et al. or "Essential COM" of Don Box for the further discussion on this topic.

Let's assume we have object B and aggregated object C (in-proc server), created by B. Can you access any interface of B from C? What's the difference between aggregated and contained objects?

Yes, you can. This is fundamental postulate of COM: "If you can get there from here, you can get there from anywhere", i.e. QI'ing for IUnknown you may proceed and to get a pointer to any other interface, supported by the object. Aggregated object exposes its interface directly, without visible intervention of the object container. Contained object is created within the object container and its interfaces might be altered or filtered by the object container.

What is ROT?GIT? Count pros and cons of both.

By definition, running object table (ROT) is a globally accessible table on each computer that keeps track of all COM objects in the running state that can be identified by a moniker. Moniker providers register an object in the table, which increments the object's reference count. Before the object can be destroyed, its moniker must be released from the table. Global Interface Table (GIT) allows any apartment (either single- or multi-threaded) in a process **to get access to an interface** implemented on an object **in any other apartment** in the process.

If you have an object with two interfaces, can you custom marshal one of them?

No! The decision to use custom marshaling is an all-or-nothing decision; an object has to custom marshal all its interfaces or none of them.

Is there a way to register in-proc server without regsvr32.exe?

Yes. Call DllRegisterServer() from the client. Do not forget to call DllUnregisterServer() from the same client. You may also use Registrar object for the same purpose or use direct manipulation of the windows registry.

What is VARIANT? Why and where would you use it?

VARIANT is a huge union containing automation type. This allows easy conversion of one automation type to another. The biggest disadvantage of VARIANT is size of the union.

How can you guarantee that only remote server is ever created by a client?

Create an object (call CoCreateObjectEx()) with CLSCTX_REMOTE_SERVER flag.

What is _declspec(notable)? Why would you need this?

[declspec](#)(notable) is a Microsoft's compiler optimization. The main idea of this optimization is to strip the vtable initialization code from abstract class (for abstract class the vtable is empty, while it is initialized in constructor).

What is an IDL?

IDL stands for Interface Definition Language. IDL is the language to describe COM interfaces.

What is In-proc?

In-proc is in-process COM object, i.e. COM object that implemented as DLL and supposed to be hosted by a container. When you have to instantiate the in-proc object remotely, you may use DLLHost.exe application that was design specially for this purpose.

What is OLE?

OLE is an object and embedding first implementation of COM spec available from MS before COM was officially named COM.

Give examples of OLE usage.

The most famous examples are probably drag and drop and structured storage implementations.

What are 2 storage types for composite document?

Storage and Stream.

Is .doc document a compound document? Is it a structured storage?

Compound document is a document that contains information about other documents hosted in this document. All office documents may be compound documents, but maybe not. Word documents from version 6.0 and up are stored as structured storage.

Difference between CoInitialize(...), and CoInitializeEx(...)?

CoInitialize(NULL);

```
CoInitializeEx(NULL, COINIT_APARTMENTTHREAD);
```

CoInitializeEx provides the same functionality as CoInitialize and also provides a parameter to explicitly specify the thread's concurrency model. CoInitialize calls CoInitializeEx and specifies the concurrency model as single-thread apartment.

Agile

Q 1. What is Agile Testing?

Ans. Agile testing is a practice that a QA follows in a dynamic environment where testing requirements keep changing according to the customer needs. It is done parallel to the development activity where testing team receives frequent small codes from the development team for testing.

Q 2. What is the difference between burn-up and burn-down chart?

Ans. Burn-up and burn-down charts are used to keep track the progress of the project.

Burn-up charts represent how much work has been completed in any project whereas Burn-down chart represents the remaining work in a project.

Q 3. Define the roles in [Scrum](#)?

Ans. There are mainly three roles that a Scrum team has:

Project Owner – who has the responsibility of managing product backlog. Works with end users and customers and provide proper requirement to the team to build the proper product.

Scrum Master – who works with scrum team to make sure each sprint gets complete on time. Scrum master ensure proper work flow to the team.

Scrum Team – Each member in the team should be self-organized, dedicated and responsible for high quality of the work.

Q 4. What is Product backlog & Sprint Backlog?

Ans. Product backlog is maintained by the project owner which contains every feature and requirement of the product.

Sprint backlog can be treated as subset of product backlog which contains features and requirements related to that particular sprint only.

Q 5. Explain Velocity in Agile?

Ans. Velocity is a metric that is calculated by addition of all efforts estimates associated with user stories completed in a iteration. It predicts how much work Agile can complete in a sprint and how much time will require to complete a project.

Q 6. Explain the difference between [traditional Waterfall model](#) and agile testing?

Ans. Agile testing is done parallel to the development activity whereas in traditional waterfall model testing is done at the end of the development.

As done in parallel, agile testing is done on small features whereas in waterfall model testing is done on whole application.

Q 7. Explain Pair Programming and its benefits?

Ans. Pair programming is a technique in which two programmer works as team in which one programmer writes code and other one reviews that code. They both can switch their roles.

Benefits:

Improved code quality: As second partner reviews the code simultaneously, it reduces the chances of mistake.

Knowledge transfer is easy: One experience partner can teach other partner about the techniques and codes.

Q 8. What is re-factoring?

Ans. Modification of the code without changing its functionality to improve the performance is called re-factoring.

Q 9. Explain the Iterative and Incremental Development in Agile?

Ans. Iterative Development: Software is developed and delivered to customer and based on the feedback again developed in cycles or release and sprints. Say in Release 1 software is developed in 5 sprints and delivered to customer. Now customer wants some changes, then development team plan for 2nd release which can be completed in some sprints and so on.

Incremental Development: Software is development in parts or increments. In each increment a portion of the complete requirement is delivered.

Q 10. How do you deal when requirements change frequently?

Ans. This question is to test the analytical capability of the candidate. Answer can be-

Work with PO to understand the exact requirement to update test cases. Also understand the risk in changing the requirement. Apart from this one should be able to write generic test plan and test cases. Don't go for the automation until requirements are finalized.

Q 11. What is a test stub?

Ans. A small code which mimics a specific component in the system and can replace it. Its output is same as the component it replaces.

Q 12. What qualities should a good Agile tester have?

Ans. (1) Agile tester should be able to understand the requirements quickly.

(2) Agile tester should know Agile concepts and principals.

(3) As requirements keep changing, he should understand the risk involved in it.

(4) Agile tester should be able to prioritize the work based on the requirements.

Communication is a must for an Agile tester as it requires a lot of communication with developers and business associates.

Q 13. What is the difference between Epic, User stories & Tasks?

Ans. User Stories: User Stories define the actual business requirement. Generally created by Business owner.

Task: To accomplish the business requirements development team create tasks.

Epic: A group of related user stories is called an Epic.

Q 14. What is a Task board in Agile?

Ans. Task board is a dashboard which shows progress of the project. It contains:

User Story: which has the actual business requirement.

To Do: Tasks that can be worked on.

In Progress: Tasks in progress.

To Verify: Tasks pending for verification or testing Done:

Completed tasks.

Q 15. What is Test Driven Development (TDD)?

Ans. It is Test-first development technique in which we add a test first before we write a complete production code. Next, we run the test and based on the result refactor the code to fulfill the test requirement.

Q 16. How QA can add a value to an agile team?

Ans. QA can provide a value addition by thinking differently about the various scenarios to test a story. They can provide quick feedback to the developers whether new functionality is working fine or not.

Q 17. What is Scrum ban?

Ans. It is a software development model which is a combination of scrum and kanban. Scrumban is considered for maintenance projects in which there are frequent changes or unexpected user stories. It can reduce the minimum completion time for user stories.

Q 18. What is Application Binary Interface?

Ans. Application Binary Interface or ABI defines an interface for compiled application programs or we can say it describes the low level interface between an application and the operating system.

Q 19. What is Zero sprint in Agile?

Ans. It can be defined as pre step to the first sprint. Activities like setting development environment, preparing backlog etc needs to be done before starting of the first sprint and can be treated as Sprint zero.

Q 20. What is Spike?

Ans. There may be some technical issues or design problem in the project which needs to be resolved first. To provide the solution of these problem “Spikes” are created. Spikes are of two types- Functional and Technical.

Q 21. Name some Agile quality strategies.

Ans. Some Agile quality strategies are-

Re-factoring

Small feedback cycles

Dynamic code analysis

Iteration

Q 22. What is importance of daily stand up meeting?

Ans. Daily stand up meeting is essential for any team in which- Team discuss about how much work has been completed.

What are the plans to resolve technical issues?

What steps need to done to complete the projects etc.

Q 23. What is tracer bullet?

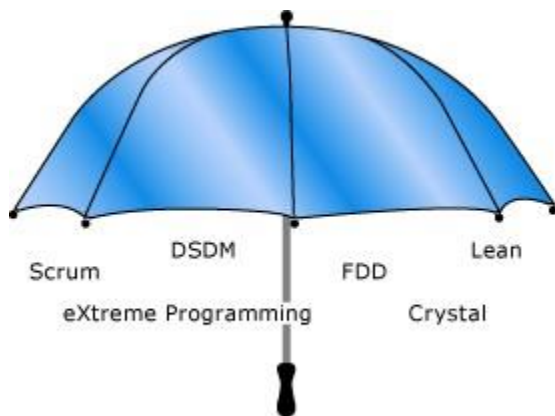
Ans. It can be defined as spike with the current architecture or the current set of best practices. The purpose of a tracer bullet is to examine how an end-to-end process will work and examine feasibility.

Q 24. How the velocity of sprint is measured?

Ans. If capacity is measured as a percentage of a 40 hours weeks then completed story points / team capacity. If capacity is measured in man hours, then Completed story points / team capacity

Q 25. What is Agile manifesto?

Ans. Agile manifesto defines an iterative and people-centric approach to software development. It has basically 4 key values and 12 principals.



Agile development is not a methodology in itself. It is an umbrella term that describes several agile methodologies.

These methodologies included Scrum, XP, Crystal, FDD, and DSDM.

SCRUM

Scrum is an iterative and incremental agile software development framework for managing software projects and product or application development.

Scrum Roles

Product Owner

Development Team

Scrum Master

Stakeholders

Managers

Product Owner

The Product Owner represents the stakeholders and is the voice of the customer.

Accountable for ensuring value to the business.

Writes (or the team) customer-centric items (user stories), prioritizes them, and adds them to the product backlog.

Scrum teams should have one, may also be a member of the development team.

Not be combined with that of the Scrum Master.

Development Team

Responsible for delivering potentially shippable product increments at the end of each Sprint.

Made up of 3–9 people with cross-functional skills who do the actual work (analyze, design, develop, test, technical communication, document, etc.).

Self-organizing, even though they may interface with project management organizations (PMOs).

Scrum Master

Accountable for removing impediments to the ability of the team to deliver the sprint goal/deliverables.

Is not the team leader, but acts as a buffer between the team and any distracting influences.

Ensures that the Scrum process is used as intended.

Enforcer of rules. Protector of the team and keep it focused on the tasks at hand.

Also been referred to as a servant-leader to reinforce these dual perspectives.

Differs from a Project Manager in that the latter may have people management responsibilities unrelated to the role of Scrum Master.

Excludes any such additional people responsibilities.

Backlog

Product Backlog:-

The product backlog is an ordered list of "requirements" that is maintained for a product. It consists of features, bug fixes, non-functional requirements, etc. - whatever needs to be done in order to successfully deliver a working software system.

Sprint Backlog:-

The sprint backlog is the list of work the Development Team must address during the next sprint. The list is derived by selecting **stories/features** from the top of the **product backlog** until the Development Team feels it has enough work to fill the sprint. This is done by the

Development Team asking "Can we also do this?" and adding stories/features to the sprint backlog.

What is the Daily Standup?

Every day, preferably in the morning, the team meets for no more than 15 minutes to answer three questions:

- 1) What did you do yesterday?
- 2) What do you plan on doing today?
- 3) Are there any blocks or impediments that keep you from doing your work?

This Scrum ceremony is not meant to be a status meeting for stakeholders, but a way to energize the team and get them to set focus for the day.

What are the ceremonies in agile?

Scrum suggests three roles: The team, ScrumMaster, and product owner;

Four ceremonies:Sprint planning meeting, Daily Scrum, sprint review meeting, and sprint retrospective meeting;

Three artifacts: The product increment, Product Backlog, and Sprint Backlog.

Recent Enhancement:

Existing Application: We are getting an XML file from external system, we have to carry the validation of that XML file.

XML file contain Document Code and Document Values. DocCode to DocValue relation is One to Many.

My Core Module is carrying this XML file validation against to the Database table (DocCodes Table).

DocCodes Table contains a list of records of

DocCodeType,DocCodeValue. Table: DocCodes

DocCodeTypeDocCodeValue

1001	BSE
1002	NYSE
1003	IRQ
1001	NSE

1003 Xyz
 custodian
1002 StateStreet
1001 NSDL

My current application is reading a record from XML file and carries validation against to database table DocCodes.

Whether the DocumentCode - DocumentValue pair existed or not.

Here, multiple times, we are connecting to the database and carrying validation.

This legacy application having performance issue, while validating the XML file, If the file size beyond 6MB.

System performance is: 13 minutes and above.

So, my team lead has given a patch to this application, to process only less than 6MB XML file.

New Enhancement: We carried this enhancement to process any size of XML file, with good performance.

We defined a cache memory object CCodeListPair, which holds therecords of DocCodes Table. Instead of carrying XML file validation to database, by firing N number of queries. We are reading the data once from DB to CCodeListPair object, when system starts. Now XML file validation is carried against to my CCodeListPair object.

Now the performance is: 6 to 11 seconds.

What is calling convention:

The arrangement of [arguments](#) for a procedure or function call. Different programming languages may require arguments to be pushed onto a [stack](#) or entered in [registers](#) in left-to-right or right-to left order, and [either](#) the caller or the callee can be responsible for removing the arguments.

The calling convention also determines if a variable number of arguments is allowed.

Types of Calling Conventions:

The following calling conventions are supported by the Visual C/C++ compiler.

Keyword	Stack cleanup	Parameter passing
cdecl	Caller	Pushes parameters on the stack, in reverse order (right to left)

__cdecl	n/a	Load parameters onto CLR expression stack in order (left to right).
__stdcall	Callee	Pushes parameters on the stack, in reverse order (right to left)
fastcall	Callee	Stored in registers, then pushed on stack
__thiscall	Callee	Pushed on stack; this pointer stored in ECX

[__cdecl](#)

This is the default calling convention for C and C++ programs. Because the stack is cleaned up by the caller, it can do **vararg** functions. The `__cdecl` calling convention creates larger executables than [stdcall](#), because it requires each function call to include stack cleanup code. The following list shows the implementation of this calling convention.

Element	Implementation
Argument-passing order	Right to left
Stack-maintenance responsibility	Calling function pops the arguments from the stack

Varargs example in c

```
#include <stdio.h>
#include <stdarg.h>
```

```
/* print all non-negative args one at a time; all
   args are assumed to be of int type */
```

```
void printargs(int arg1, ...)
```

```
{
```

```
    va_list ap;
    int i;
```

```
    va_start(ap, arg1);
```

```
        for (i = arg1; i >= 0; i = va_arg(ap, int))
            printf("%d ", i);
```

```
    va_end(ap);
    putchar('\n');
```

```
}
```

```
int main(void)
```

```
{
```

```

printargs(5, 2, 14, 84, 97, 15, 24, 48, -1);
printargs(84, 51, -1); printargs(-1);
printargs(1, -1); return 0;
}

```

This program yields the output:

5 2 14 84 97 15 24 48 84

51

1

_stdcall

The **_stdcall** calling convention is used to call Win32 API functions. The callee cleans the stack, so the compiler makes **vararg** functions **_cdecl**. Functions that use this calling convention require a function prototype.

return-type_stdcall function-name[(argument-list)]

The following list shows the implementation of this calling convention.

Element	Implementation
Argumentpassing order	Right to left.
Argumentpassing convention	By value, unless a pointer or reference type is passed.
Stack-maintenance responsibility	Called function pops its own arguments from the stack.
Name-decoration convention	An underscore (<code>_</code>) is prefixed to the name. The name is followed by the at sign (<code>@</code>) followed by the number of bytes (in decimal) in the argument list. Therefore, the function declared as <code>intfunc(int a, double b)</code> is decorated as follows: <code>_func@12</code>

_fastcall

The **_fastcall** calling convention specifies that arguments to functions are to be passed in registers, when possible. The following list shows the implementation of this calling convention.

Element	Implementation
Argumentpassing order	The first two DWORD or smaller arguments are passed in ECX and EDX registers; all other arguments are passed right to left.
Stackmaintenance responsibility	Called function pops the arguments from the stack.
Name-decoration convention	At sign (@) is prefixed to names; an at sign followed by the number of bytes (in decimal) in the parameter list is suffixed to names.

SQL

1. Explain DDL, DML, DCL and TCL commands?

DDL (Data Definition Language): DDL or Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in database.

Examples of DDL commands:

- ✦ **CREATE** – is used to create the database or its objects (like table, index, function, views, store procedure and triggers).
- ✦ **DROP** – is used to delete objects from the database.
- ✦ **ALTER** – is used to alter the structure of the database.
- ✦ **TRUNCATE** – is used to remove all records from a table, including all spaces allocated for the records are removed.
- ✦ **COMMENT** – is used to add comments to the data dictionary.
- ✦ **RENAME** – is used to rename an object existing in the database.

DML (Data Manipulation Language): The SQL commands that deals with the manipulation of data present in database belong to DML or Data Manipulation Language and this includes most of the SQL statements.

Examples of DML:

- ✦ **SELECT** – is used to retrieve data from the database.
- ✦ **INSERT** – is used to insert data into a table.
- ✦ **UPDATE** – is used to update existing data within a table.
- ✦ **DELETE** – is used to delete records from a database table.

DCL (Data Control Language): DCL includes commands such as GRANT and REVOKE which mainly deals with the rights, permissions and other controls of the database system.

Examples of DCL commands:

- ✦ **GRANT** – gives user's access privileges to database.
- ✦ **REVOKE** – withdraw user's access privileges given by using the GRANT command.

TCL (transaction Control Language): TCL commands deals with the transaction within the database.

Examples of TCL commands:

- ✦ **COMMIT** – commits a Transaction.
- ✦ **ROLLBACK** – rollbacks a transaction in case of any error occurs.
- ✦ **SAVEPOINT** – sets a savepoint within a transaction.
- ✦ **SET TRANSACTION** – specify characteristics for the transaction.

2. What's the difference between a primary key and a unique key?

Primary Key	Unique Key
-------------	------------

Both enforces uniqueness of the column on which they are defined.	
Creates clustered index on the column	No clustered index by default.
Doesn't allows NULL	Allows one NULL only.

3. What are the difference between clustered and a non-clustered index?

- **A clustered index** is a special type of index that reorders the way records in the table are physically stored. Therefore, table can have only one clustered index. The leaf nodes of a clustered index contain the data pages.
- **A non-clustered index** is a special type of index in which the logical order of the index does not match the physical stored order of the rows on disk. The leaf node of a non-clustered index does not consist of the data pages. Instead, the leaf nodes contain index rows.

4. What is difference between DELETE and TRUNCATE commands?

Delete command removes the rows from a table based on the condition that we provide with a WHERE clause.

Truncate will actually remove all the rows from a table and there will be no data in the table after we run the truncate command.

TRUNCATE:

- TRUNCATE is faster and uses fewer system and transaction log resources than DELETE.
- TRUNCATE removes the data by de-allocating the data pages used to store the table's data, and only the page de-allocations are recorded in the transaction log.
- TRUNCATE removes all rows from a table, but the table structure, its columns, constraints, indexes and so on, remains. The counter used by an identity for new rows is reset to the seed for the column.
- You cannot use TRUNCATE TABLE on a table referenced by a FOREIGN KEY constraint.
- Because TRUNCATE TABLE is not logged, it cannot activate a trigger.
- TRUNCATE cannot be rolled back.
- TRUNCATE is DDL Command.
- TRUNCATE Resets identity of the table

DELETE:

- DELETE removes rows one at a time and records an entry in the transaction log for each deleted row.
- If you want to retain the identity counter, use DELETE instead. If you want to remove table definition and its data, use the DROP TABLE statement.
- DELETE Can be used with or without a WHERE clause
- DELETE Activates Triggers.
- DELETE can be rolled back.
- DELETE is DML Command.
- DELETE does not reset identity of the table.

Note: DELETE and TRUNCATE both can be rolled back when surrounded by TRANSACTION if the current session is not closed. If TRUNCATE is written in Query Editor surrounded by TRANSACTION and if session is closed, it cannot be rolled back but DELETE can be rolled back.

5. What is the difference between a HAVING CLAUSE and a WHERE CLAUSE?

They specify a search condition for a group or an aggregate.

But the difference is that HAVING can be used only with the SELECT statement. HAVING is typically used in a GROUP BY clause. When GROUP BY is not used, HAVING behaves like a WHERE clause.

Having Clause is basically used only with the GROUP BY function in a query whereas WHERE Clause is applied to each row before they are part of the GROUP BY function in a query

6. What is PRIMARY KEY?

A PRIMARY KEY constraint is a unique identifier for a row within a database table. Every table should have a primary key constraint to uniquely identify each row and only one primary key constraint can be created for each table. The primary key constraints are used to enforce entity integrity.

7. What is UNIQUE KEY constraint?

A UNIQUE constraint enforces the uniqueness of the values in a set of columns, so no duplicate values are entered. The unique key constraints are used to enforce entity integrity as the primary key constraints.

8. What is FOREIGN KEY?

A FOREIGN KEY constraint prevents any actions that would destroy links between tables with the corresponding data values. A foreign key in one table points to a primary key in another table. Foreign keys prevent actions that would leave rows with foreign key values when there are no primary keys with that value. The foreign key constraints are used to enforce referential integrity.

9. What is CHECK Constraint?

A CHECK constraint is used to limit the values that can be placed in a column. The check constraints are used to enforce domain integrity.

10. What are the advantages of using Stored Procedures?

- ✦ Stored procedure can reduce network traffic and latency, boosting application performance.
- ✦ Stored procedure execution plans can be reused, staying cached in SQL Server's memory, reducing server overhead.
- ✦ Stored procedures help promote code reuse.
- ✦ Stored procedures can encapsulate logic. You can change stored procedure code without affecting clients.
- ✦ Stored procedures provide better security to your data.

11. Differences between Functions and Stored Procedures?

Stored Procedures are pre-compile objects which are compiled for first time and its compiled format is saved which executes (compiled code) whenever it is called. But Function is compiled and executed every time when it is called.

Basic Difference

- ✦ Function must return a value but in Stored Procedure it is optional (Procedure can return zero or n values).
- ✦ Functions can have only input parameters for it whereas Procedures can have input/output parameters.
- ✦ Functions can be called from Procedure whereas Procedures cannot be called from Function.

Advance Difference

- ✦ Procedure allows SELECT as well as DML (INSERT/UPDATE/DELETE) statement in it whereas Function allows only SELECT statement in it.
- ✦ Procedures cannot be utilized in a SELECT statement whereas Function can be embedded in a SELECT statement.
- ✦ Stored Procedures cannot be used in the SQL statements anywhere in the WHERE/HAVING/SELECT section whereas Function can be.
- ✦ Functions that return tables can be treated as another rowset. This can be used in JOINS with other tables.
- ✦ Inline Function can be thought of as views that take parameters and can be used in JOINS and other Rowset operations.
- ✦ Exception can be handled by try-catch block in a Procedure whereas try-catch block cannot be used in a Function.
- ✦ We can go for Transaction Management in Procedure whereas we can't go in Function.

12. What is a Trigger?

Triggers are used to execute a batch of SQL code when insert or update or delete commands are executed against a table. Triggers are automatically triggered or executed when the data is modified. It can be executed automatically on insert, delete and update operations.

What are the types of Triggers?

There are two types of triggers

- ✦ **AFTER Triggers:** AFTER Triggers executed following the SQL actions, such INSERT, UPDATE or DELETE.
- ✦ **INSTEAD OF Triggers:** for example, If an INSTEAD OF UPDATE trigger exists on Sales Table, and an UPDATE statement is executed against the Sales TABLE, The UPDATE statement will not change a row in the Sales Table. Instead the UPDATE statement causes the INSTEAD OF trigger to be executed.

INSTEAD OF triggers are executed automatically before the primary key, and the foreign key constraints are checked, whereas the traditional AFTER triggers are executed after these constraints are checked.

Unlike AFTER triggers, INSTEAD OF triggers can be created on views.

13. What is the difference between UNION and UNION ALL?

- ✦ **UNION:** To select related information from two tables UNION command is used. It is similar to JOIN command.
- ✦ **UNION All:** The UNION ALL command is equal to the UNION command, except that UNION ALL selects all values. It will not remove duplicate rows, instead it will retrieve all rows from all tables.

14. What is the difference between COMMIT and ROLLBACK?

Every statement between BEGIN and COMMIT becomes persistent to database when the COMMIT is executed. Every statement between BEGIN and ROLLBACK are reverted to the state when the ROLLBACK was executed.

15. Explain about View in SQL?

A view is a query stored as an object in the database. View is a virtual table, it doesn't contain any data or information. A view is only able to hold one query at a time.

Why we are creating Views?

- There is often a requirement to write complex joins, aggregates, functions and other T_SQL functions to produce the results.
- To mask this complexity, this statement can be created as a view, and then future queries can be performed against the view.

SQL Server 2000 has a new feature called the View Index, it is similar to Oracle's materialized view. SQL Server View Indexes give the capability to define index on View. Changes in the base tables are automatically reflected in the index view.

Reference Link: <https://www.codeproject.com/Articles/15609/Views-in-SQL>

16. What is cursor?

A cursor is a temporary work area created in system memory when an SQL statement is executed.

A cursor is a set of rows together with a pointer that identifies a current row. It is a database object to retrieve data from a result set one row at a time. It is useful when we want to manipulate the record of a table in a singleton method, in other words one row at a time. In other words, a cursor can hold more than one row, but can process only one row at a time. The set of rows the cursor holds is called the active set.

Types of Cursors

There are the following two types of Cursors:

- Implicit Cursor
- Explicit Cursor

Implicit Cursor: - These types of cursors are generated and used by the system during the manipulation of a DML query (INSERT, UPDATE and DELETE). An implicit cursor is also generated by the system when a single row is selected by a SELECT command.

Explicit Cursor: - This type of cursor is generated by the user using a SELECT command. An explicit cursor contains more than one row, but only one row can be processed at a time. An explicit cursor moves one by one over the records. An explicit cursor uses a pointer that holds the record of a row. After fetching a row, the cursor pointer moves to the next row.

Main components of Cursors

Each cursor contains the followings 5 parts:

Declare Cursor: In this part we declare variables and return a set of values.

Open: This is the entering part of the cursor.

Fetch: Used to retrieve the data row by row from a cursor.

Close: This is an exit part of the cursor and used to close a cursor.

Deallocate: In this part we delete the cursor definition and release all the system resources associated with the cursor.

17. How to delete duplicate rows in SQL Server?

Duplicate rows can be deleted using CTE and ROW NUMER feature of SQL Server.

How to delete duplicate rows from table, without primary key.

//// to do ///

18. What is connection pooling? /// TO DO

<https://www.codeproject.com/Articles/17768/ADO-NET-Connection-Pooling-at-a-Glance>

19. What is temp table?

SQL Joins

INNER JOIN: Inner join shows matches only when they exist in both tables.

Example... I have two tables Customers, and Orders.

Inner join made on **Customers.CustomerID** and **Orders.CustomerID**

Result: SQL result is Customers who have orders, If the customers doesn't have order, it will not be display the record.

Query:

```
SELECT Customer.*, Orders.* FROM Customer INNER JOIN Orders  
ON Customer.CustomerID = Orders.CustomerID;
```

LEFT OUTER JOIN: Left join will display all records in left table of the SQL

Statement. Example... I have two tables Customers, and Orders.

Result:

1. Customers with or without orders will be displayed.
2. Orders data for customers without orders appear as NULL values

Here we can determine the amount ordered by each customer and we need to see who has not ordered anything as well.

Query:

```
SELECT Customer.*, Orders.* FROM Customer LEFT OUTER JOIN  
Orders ON Customer.CustomerID = Orders.CustomerID;
```

RIGHT OUTER JOIN: Right join will display all records in the right table of the SQL

Statement. Example... I have two tables Customers, and Orders.

Result:

1. All orders with or without matching customer records will be displayed.
2. Customers data for orders without customers appear as NULL values.

Here we want to determine, if there are any orders in the data with undefined **CustomerID** values. RIGHT OUTER JOIN is a mirror image of LEFT OUTER JOIN.

Query:

```
SELECT Customer.*, Orders.* FROM Customer RIGHT OUTER  
JOIN Orders ON Customer.CustomerID = Orders.CustomerID;
```

Frequently Asked Queries

1. Find Nth Highest Salary from a table.

```
SELECT Salary from
Employee e1 WHERE 3 = (
    SELECT COUNT( DISTINCT(e2.Salary)) From
    Employee e2 WHERE e2.Salary >=
    e1.Salary
);
```

Explanation: -

- Take the salary one by one by outer query and pass it to the inner query.
- Count those salaries which are greater than equal to that salary (passed by outer query)
- The inner query sends the count to the outer query and matches it with the digit we obtained from the outer query like 3 because we want to find the 3rd highest salary.

Corporate Action Project

Corporate Actions: - I Involved in this application for managing a corporate action, So, A corporate action is an accounting event, initiated by the issuer of a specific security.

It is developed on MFC Doc/View architecture, and it is an MDI application (Multiple Document Interface). Here I developed various dialog boxes, and control customization using MFC.

I Developed dialog screens, for corporate actions like...

Stock Split, Taxable Exchange, Spin Off, Stock Dividend, Bonus Issue, Warrant, and Interest Capitalization...

I developed business objects, which are serializable for all the corporate actions I mentioned. So, here I involved in stock split & spin offs for multiple funds & individual funds...

I developed document objects, which can be displayed with multiple views... for example.

CMultiFundView, and CIndividualFundView.

and I Involved in Configurable or Customizable toolbars, for country specific Taxable and Tax-free exchange for securities.

In C++ I implemented OOPS concepts like abstraction, encapsulation, inheritance and Polymorphism and followed SOLID Principles (Single Responsibility, Open/Close, Liskov substitution, Interface segregation and Dependency Inversion principle).

CCorporateAction is an abstract base class, which is implemented by various concrete objects like Stock Split, Taxable Exchange, and Spin Off.

I implemented Runtime Polymorphism for generating the reports. CReports is my base class for all my report objects.

My legacy application was developed in C++, and MFC, now we need to integrate it with .NET Core Application, where we developed a managed DLL, Here I involved in marshalling and unmarshalling.

Here I also involved in writing C# restful services using Asp.NET Core3.1 Web API.

RKS Financial Project

Comarch Custody is a complex portfolio asset management system that records and accounts for all investments in investment portfolio, and interfaces with other systems for data entry and data export.

With Comarch Custody we can do ...

- ✦ Set up security information
- ✦ Enter transactions
- ✦ Viewing position information ✦ Reconcile balances with the custodian ✦ -Run and view reports.

We can find two kinds of applications in stock trading.

Front Office: Here we have some web portals, we are submitting trades to the stock exchanges and those trades will be executed.

Back Office: Executed trades will be sent for the settlement to the clearing house, **so my application is back-office application which works with clearing house.**

Clearing house core operations are:

- ✦ Trade Capture
- ✦ Trade Enrichment
- ✦ Trade Validation,
- ✦ Trade Matching
- ✦ Trade Confirmation.

This application is for Investment managers/Fund managers, they are doing bulk trades settlement. It's not for a single person to do the trade.

This application is an MDI application, so we can view all the portfolios/Issues/Issuer's in a grid view.

We can track all the positions (posted trades either purchase/Sale). We can do modify/suspend the trades.

Here I involved in various Modules:

- ✦ TADLG (Trade Action Dialogs screens, which is UI module)
- ✦ Fixed Income securities (it is business objects)
- ✦ Derivates (it is also business objects)
- ✦ Reporting (which is generating csv reports)

My Roles & Responsibilities in this project:

- ✦ Implemented View Objects to track the current status of all reconciliations.
- ✦ Implemented Dialog, and View screens to offer a series of dashboards to business users. So, users can view and validate all real-time and historic data from the reconciliation repository.
- ✦ Implemented various Document, and View objects for each trade position. ✦ Implemented settlement notifications for reconciliation.
- ✦ UI screens modified in stock dividends.
- ✦ Implementation of Taxable & Tax-Free Exchange for securities.
- ✦ While developing business objects, I implemented Abstraction, Encapsulation, Inheritance and Polymorphism concepts.

Technically I involved in developing Modal/Modeless Dialog screens, DLL's Regular and Extension, and also COM DLLs which is InProc Server.

In C++ 11, I used smart pointers (unique/shared/weak pointers), LAMBDA expressions, STL containers like vector, list, map objects. (yeah that's it from my side).

The trading operations will be carried against portfolio account. Before posting any trade...

Create a portfolio account and configure the portfolio account with cash accounts and custodian account cash accounts are two types.

- 1. Capital Account and**
- 2. Income Account**

Setup Issue and Issuer account. Issue is any investment bond and Issuer is the investor.

My application is processing 14 kinds of investment bonds.

- ✦ Fixed Income
- ✦ Money market
- ✦ Open Repo
- ✦ Term repo
- ✦ Preferred Stock
- ✦ Common Stock
- ✦ Partnership
- ✦ Options
- ✦ Rights
- ✦ Warrants
- ✦ Futures
- ✦ TBA
- ✦ Swaps

After setting up portfolio with bank/custodian accounts, issue and issuer accounts. anyone can do trading (posting a trade) against portfolio account.

for ex: in my application an issue(bond) posted to sale by an issuer(investor) from a portfolio account.

Environment: MFC MDI Application, some modules already developed in Win32 API, and this application is integrated with .NET Framework application, which is developed C#, WPF, REST, SQL Server

Integration is done by the Managed C++ Library (Here I involved)

Tools: Visual Studio 2015 (earlier we used Visual Studio 2008), SQL Server 2012, Case tracker (bug tracking tool), Team Foundation Server (for AGILE), Tortoise SVN, SVN repository for internal releases.

Intex Project (cmosub32 DLL)

My application is a structured finance application It's all about home equities/ home loans. I involved in this application on risk analysis. So, here I involved in prepay risk, loss risk and severity risk of tranches and collaterals.

- **Tranches are investment bonds and**
- **collaterals are the loans.**

We have a library cmosub32 API which loads the deal file. Here deal files are the flat files, which can be loaded by the cmosub32 API only.

FNMA/GNMA are issuing the deals. A deal has some capital amount, which is collected from the retail investors. This deal amount will be distributed to the people, who had applied for the loans.

A deal is a collection of tranches. Here tranche is the fixed income bond which is traded at stock exchanges in the US market.

A deal is a collection of loans.

People are repaying their loan amount to the deal. Deal is repaying to the investors (Tranches).

1. Suppose people are paying their loan amount before their tenure. Here deal will suffer from the **risk of prepayment**.
2. Suppose people are default, while repaying their money. Here deal will suffer the **default/loss risk** 3. Suppose people are doing balloon payments, here deal will suffer the **severity risk**.

cmosub32 is a 3rd party "C" language library. Our input files are .cdi files, and .cdi files for loading the deal files. The deal information is an encrypted .cdi file

- **CDI** means collateral deal information
- **CDU** means collateral deal updated

for cashflow projections we are parsing prepay/loss/severity ramp files., and also interest rates files.

This application is developed on MFC Doc/View Architecture, using MDI (Multiple Document Interface) template. Here deal information will be displayed as 2 or 3 views. like detailed deal information and summary information. In this application, I involved in the development of modal/modeless dialog boxes, and also in the development of

MFC GUI control's customization/subclassing like edit and list boxes.

I had written lot of c++ code using OOPS concepts. While writing the deal class, I implemented abstraction and encapsulation.

While developing the reports, I had implemented Inheritance and polymorphism. In the development of Tranche and Cashflow classes, I had implemented composition and aggregation.

C++ 11 features are used in this project, like smart pointers (unique_ptr, shared_ptr, weak_ptr), LAMBDA expressions, for_each algorithms, range based for loops, STL containers like vector, list, and map objects.

Here I implemented design patterns like singleton, bridge, observer, and factory. Our configuration settings are

implemented with thread-safe singleton class.

Coming to Windows Programing using MFC, I had developed modal and modeless dialog boxes. I had developed a

CUSIP (Common Uniform Security Identification Procedure) search modeless dialog box which uses worker thread. For the last one year, I Involved in writing RESTful services using Asp.NetCore 3.1.

Here I involved in coding for service objects, which are injected into the Controller class, using dependency injection.

These services are accessing Managed Dll's, where we had written Unmanaged + Managed code, the unmanaged code, which is our Legacy or Native Application.

1. How can you load a deal file? YyyyMmDd

```
settle_date ;
char          *deal_name = "AAST0503";
ICMO          *icmop ;
ICMOMISC      *icmomiscp ;

settle_date = icmo_YyyyMmDd_today();
icmomiscp = (ICMOMISC *)icmo_make_struct( NULL, ICMOMS_ICMOMISC, "TYPICAL",
NULL ) ;
icmomiscp->icmomisc_trading_accuracy_not_required = TRUE ; icmomiscp-
>icmomisc_alteruse.altu_base_cdi_path = CDI_PATH ; icmomiscp-
>icmomisc_alteruse.altu_base_cdu_path = CDU_PATH ; icmomiscp-
>icmomisc_accept_extended_index_list = TRUE ;

icmop = icmo_deal( icmomiscp, NULL, deal_name, ICMODEAL_SEASONED_POOLS,
settle_date, 0, 0, NULL, NULL ) ;
```

// If deal file is loaded successfully, then it returns ICMO struct //
pointer, else returns NULL.

2. How to get tranche number?

```
Int trno = -1;
```

```
trno = icmo_tranche_num (icmop, "COLLAT"); // Here "Collat" is the tranche name.
```

3. How to get all tranche names?

```
for (int trno = icmop->icmo_have_residul ? 0: 1;
trno <= icmo_n_tranches;
trno++)
{
printf ("\n %s", icmop->icmo_tranche_names[trno]);
}
```

4. Explain some memory leak scenario's in your project?

A deal file (flat file) is loaded by calling **icmo_deal(...)**, it creates ICMO structure object dynamically, and returns ICMO struct pointer, ICMO pointer is needed to call CMOSUB32 API's.

After processing the deal information...

We need to deallocate the ICMO struct object, which is created on HEAP.

```
icmo_free(icmop); // it deallocates ICMO structure.
```

5. How to generate cashflows(returns) for a tranche (investment bond)?

```
int trno = icmo_tranenum (icmop, "G"); // Here "G" is the tranche name.
```

```
// configure cashflow settings.
```

```
icmo_cashflows( icmop, 0L, ICMOSPDV_VCP, 1, vspeeds,  
                NULL, NULL, ICMOCFS_MEMSTORE, NULL ); // it generates cashflows for all  
                //tranches in memory.
```

```
icmo_stats( icmop, trno, 0., ICMOPY_NONE, 0. );
```

```
for ( ith_month = 0; ith_month <= icmop->icmot_matures ;ith_month++ )  
{  
    if ( icmop->icmot_vectors[ICMOTV_principal] != NULL &&          icmop->  
        icmot_vectors[ICMOTV_interest] != NULL &&          icmop->icmot_vectors[ICMOTV_cashflow]  
        != NULL &&          icmop->icmot_vectors[ICMOTV_balance] != NULL )  
    {  
        fprintf(fp, "%d,%s,%0.2f,%0.2f,%0.2f,%0.2f\n", ith_month,  
                icmo_tr_fmtdate( icmop, trno, ith_month, ICMOFMT_YYYYMMDD ),  
                icmop->icmot_vectors[ICMOTV_principal][ith_month],  
                icmop->icmot_vectors[ICMOTV_interest][ith_month],          icmop->  
                icmot_vectors[ICMOTV_cashflow][ith_month],          icmop->  
                icmot_vectors[ICMOTV_balance][ith_month] );  
    }  
}
```

6. What is your role in your project?

I'm carrying risk analysis for the tranches (investment bonds)

Tranches are suffering with 3 types of risks. a)

Prepay risk

b) Loss risk (Default risk)

c) Severity risk.

Q1) How you are doing prepay risk analysis?

1. We have to define prepay_risk(...) callback function, which is passed as an argument to icmo_cashflows(...) function.

Here this prepay_risk(...) callback function is called for each pool in the deal.

(Here a pool is a collection of loans).

A tranche is a composition of pools. extern "C" void user_prepay_fcn(ICMO
*icmop, POOL_INFO *pip, int ith_pool, int curmonth, int n_speeds, double
*speed_vect)

```
{  
    // iterating through all pools, and assigning some assumption values, to each pool.  
}
```

```
void main( )
```

```
{  
    double      vprepays[MAX_MONTHS+1];  
    double      vdefaults[MAX_MONTHS+1];  
    double      vseverity[MAX_MONTHS+1];
```

```

vprepays[0] = 0.0;
for(int i = 1; i <= MAX_MONTHS; i++)
    vprepays[i] = 5.0;

vdefaults[0] = 0.0;    for(int i = 1; i
<= MAX_MONTHS; i++)
    vdefaults[i] = 3.0;

vseverity[0] = 0.0;
for(int i = 1; i <= MAX_MONTHS; i++)
    vseverity[i] = 100.0;

icmop->icmo_cfassum.cfa_loss_type = ICMODEFAULT_VCDR;

icmop->icmo_cfassum.cfa_loss_n_defaults = MAX_MONTHS;    icmop-
>icmo_cfassum.cfa_loss_vdefaults = vdefaults;    icmop->icmo_cfassum.cfa_loss_n_severity
= MAX_MONTHS;    icmop->icmo_cfassum.cfa_loss_vseverity = vseverity;

    icmo_cashflows(icmop,0L, ICMOSPDT_VCPR, MAX_MONTHS,vprepays,
                    user_prepay_fcn,NULL,ICMOCFS_MEMSTORE,NULL);
    trno = icmo_tranchenum(icmop,"collat");
    icmo_stats(icmop,trno, 0.0, ICMOPY_NONE, 0.0);
}

```

7. What are the data structures, used in your project?

POOL_INFO is a structure, which is maintained as a linked list.
 POOL_INFO structure, is a collection of pools, each pool has some loans.
 POOL_INFO linked list head pointer is a member variable of ICMO structure.

Here, I'm iterating through all the POOL_INFO objects, using a loop.

```

// sample code POOL_INFO
*pip = NULL;
for(pip = icmop->icmo_collat_list; pip != NULL; pip = (POOL_INFO *)pip->pi_next)
{
    cout<<"Pool ID: "<< pip->pi_pnum <<endl;
    cout<<"Original Balance: "<< pip->pi_origbal <<endl;
}

```

// Todo: Loss Risk, and Severity Risk.

8. Explain your involvement in Reports module?

I involved in generating the following reports.

1. Deal Summary
2. Tranche Summary
3. Deal, Tranche, and Collateral Cashflows
4. Strats and Trigger information Reports
5. Calculate NAV
6. Cash position with valuation
7. Current positions by market value

8. Daily valuation report
9. Fee income report
10. Fixed Income analysis report.